

Université de La Réunion

École Supérieure d'Ingénieurs Réunion Océan Indien (ESIROI)

Rapport de Travaux Pratiques

Supervision d'une infrastructure TAAF avec Grafana

Métriques systèmes et données métier

Étudiant : Mathias ALY BERIL
Formation : 4A Informatique
Année universitaire : 2025-2026

Module : Supervision de la sécurité

Encadrant : Thibaut Fontaine

Saint-Pierre de La Réunion

Contents

1	Introduction	7
1.1	Validation de la connectivité entre les machines virtuelles	7
1.2	Vérification des règles de pare-feu	7
1.3	Validation des services Docker sur la VM Monitoring	7
1.4	Test d'accessibilité des services via IP fixe	8
1.5	Validation de l'accès à PostgreSQL depuis la VM Monitoring	8
1.6	Vérification du réseau Docker interne	8
2	TP1 – Grafana : Dashboard de Supervision des Conteneurs TAAF	9
2.1	Question 1 – Variables de base (Facile)	9
2.1.1	Question	9
2.1.2	Méthode	9
2.1.3	Résultat	9
2.2	Question 2 – Métriques simples (Facile)	10
2.2.1	Question	10
2.2.2	Requête PromQL	10
2.2.3	Résultat	10
2.3	Question 3 – Filtrage par conteneur (Facile)	10
2.3.1	Question	10
2.3.2	Requête PromQL	10
2.3.3	Résultat et analyse	10
2.4	Question 4 – Calcul de pourcentage CPU (Moyen)	12
2.4.1	Question	12
2.4.2	Requête PromQL	12
2.4.3	Résultat et interprétation	12
2.5	Question 5 – Analyse temporelle mémoire (Moyen)	13
2.5.1	Question	13
2.5.2	Requête PromQL	13
2.5.3	Résultat et analyse	13
2.6	Question 6 – Métriques réseau (Moyen)	14
2.6.1	Question	14
2.6.2	Requête PromQL	14
2.6.3	Résultat et analyse	14
2.7	Question 7 – Corrélation métrique (Difficile)	15
2.7.1	Question	15
2.7.2	Conteneur analysé	15
2.7.3	Requêtes PromQL utilisées	15
2.7.4	Analyse de corrélation	16
2.7.5	Conclusion	16
2.8	Question 8 – Alertes de seuil mémoire (Difficile)	16
2.8.1	Question	16
2.8.2	Requête PromQL	16
2.8.3	Résultats observés	17

2.8.4	Interprétation	17
2.9	Question 9 – Performance système / I/O disque (Difficile)	17
2.9.1	Question	17
2.9.2	Requête PromQL	17
2.9.3	Analyse des résultats	17
2.9.4	Impact opérationnel	19
2.10	Question 10 – Analyse et propositions d’amélioration (Avancé)	19
2.10.1	Amélioration proposée : suivi des redémarrages de conteneurs	19
2.10.2	Amélioration proposée : alerte mémoire proactive	20
3	TP 2 — Dashboards de Supervision TAAF (Données Métier)	21
3.1	Objectif du TP	21
3.2	Question 1 — Données de base (Facile)	21
3.2.1	Énoncé	21
3.2.2	Requête SQL	21
3.2.3	Analyse	21
3.2.4	Résultat	21
3.3	Question 2 — Filtrage simple (Facile)	22
3.3.1	Énoncé	22
3.3.2	Requête SQL	22
3.3.3	Analyse	22
3.3.4	Résultat	22
3.4	Question 3 — Personnel actuel (Facile)	23
3.4.1	Énoncé	23
3.4.2	Requête SQL	23
3.4.3	Analyse	23
3.4.4	Résultat	24
3.5	Question 4 — Calcul de moyenne (Moyen)	24
3.5.1	Énoncé	24
3.5.2	Requête SQL	25
3.5.3	Analyse	25
3.5.4	Résultat	25
3.6	Question 5 — Analyse par type de communication (Moyen)	26
3.6.1	Énoncé	26
3.6.2	Requête SQL	26
3.6.3	Analyse	27
3.7	Question 6 — Analyse temporelle (Moyen)	28
3.7.1	Énoncé	28
3.7.2	Requête SQL	28
3.7.3	Analyse	29
3.8	Question 7 — Personnel en mission longue (Difficile)	29
3.8.1	Énoncé	29
3.8.2	Requête SQL	29
3.8.3	Analyse	30
3.9	Question 8 — Analyse des équipements critiques (Difficile)	30

3.9.1	Énoncé	30
3.9.2	Requête SQL	31
3.9.3	Analyse	31
3.10	Question 9 — Dashboard d’alertes globales (Difficile)	33
3.10.1	Énoncé	33
3.10.2	Analyse	33
3.11	Question 10 — Optimisation opérationnelle (Expert)	33
3.11.1	Analyse et propositions	33
3.12	Conclusion	33
4	TP3 — Loki Integration : Logs & Métriques TAAF	34
4.1	1. Configuration réalisée	34
4.2	2. Dashboards créés	34
4.3	3. Alertes hybrides	35
4.4	4. Analyse d’incident	35
4.5	5. Apprentissages	35
5	TP3 – Exercices pratiques : Logs & Métriques	36
5.1	Exercice 1 – Investigation d’incident guidée	36
5.1.1	Contexte	36
5.1.2	Démarche d’analyse	36
5.1.3	Analyse et conclusion	37
5.2	Exercice 2 – Dashboard personnalisé (Intermédiaire)	37
5.2.1	Objectif	37
5.2.2	Contenu du dashboard	37
5.2.3	Intérêt opérationnel	37
5.3	Exercice 3 – Alerte prédictive (Avancé)	37
5.3.1	Objectif	37
5.3.2	Principe de détection	37
5.3.3	Résultat observé	38
5.4	Réponses aux questions de validation	38
5.5	Question 1 — Configuration de base (Facile)	38
5.5.1	Question	38
5.5.2	Réponse	38
5.6	Question 2 — Requête LogQL simple (Facile)	38
5.6.1	Question	38
5.6.2	Requête LogQL	38
5.6.3	Explication	39
5.6.4	Résultat observé	39
5.7	Question 3 — Corrélation simple (Facile)	39
5.7.1	Question	39
5.7.2	Méthode	39
5.7.3	Requête PromQL (CPU)	40
5.7.4	Requête LogQL (erreurs)	40
5.7.5	Analyse attendue	40

5.8	Question 4 — Requête Prometheus (Moyen)	40
5.8.1	Question	40
5.8.2	Requête PromQL	40
5.8.3	Explication	40
5.9	Question 5 — Analyse de patterns (Moyen)	41
5.9.1	Question	41
5.9.2	Requête LogQL (principe)	41
5.9.3	Explication	41
5.10	Question 6 — Dashboard hybride (Moyen)	41
5.10.1	Question	41
5.10.2	Configuration du panel (Time series à deux requêtes)	42
5.10.3	Requête A — CPU (Prometheus)	42
5.10.4	Requête B — Volume d’erreurs (Loki)	42
5.10.5	Analyse attendue	42
5.11	Question 7 — Configuration d’alerte (Moyen)	43
5.11.1	Question	43
5.11.2	Règle d’alerte (YAML)	43
5.11.3	Explication	43
5.11.4	Test de déclenchement (simulation)	43
5.12	Question 8 — Investigation d’incident (Difficile)	43
5.12.1	Question	43
5.12.2	Démarche d’analyse	44
5.12.3	Requêtes typiques (exemples)	44
5.12.4	Analyse attendue	44
5.13	Question 9 — Debug et simulation (Moyen)	44
5.13.1	Question	44
5.13.2	Commande de simulation	44
5.13.3	Détection côté métriques	44
5.13.4	Analyse côté logs	45
5.13.5	Analyse attendue	45
5.14	Question 10 — Synthèse et cas d’usage TAAF (Difficile)	45
5.14.1	Question	45
5.14.2	Analyse comparative	45
5.14.3	Cas d’usage 1 — Dégradation des communications applicatives	45
5.14.4	Cas d’usage 2 — Saturation PostgreSQL (connexions / verrous)	45
5.14.5	Cas d’usage 3 — Incidents applicatifs Nextcloud (HTTP 5xx)	45
5.15	Problèmes rencontrés	46
5.15.1	1) Loki Ruler : règles non chargées / répertoire inattendu	46
5.15.2	2) Grafana : conflit provisioning / datasources persistantes	46
6	TP4 — Alerting TAAF	47
6.1	Réponses aux questions de validation	47
6.2	Question 1 — Configuration de base (Facile)	47
6.2.1	Question	47
6.2.2	Réponse	47

6.3	Question 2 — Webhook simple (Facile)	48
6.3.1	Question	48
6.3.2	Réponse	48
6.4	Question 3 — Test d’alerte (Facile)	48
6.4.1	Question	48
6.4.2	Méthode utilisée	48
6.4.3	Résultat observé	48
6.5	Question 4 — Seuils d’alerte (Moyen)	49
6.5.1	Question	49
6.5.2	Justification technique	49
6.5.3	Impact opérationnel	49
6.6	Question 5 — Expression PromQL (Moyen)	49
6.6.1	Question	49
6.6.2	Règle utilisée	50
6.7	Question 6 — Routing d’alertes (Moyen)	50
6.7.1	Question	50
6.7.2	Réponse	50
6.8	Question 7 — Alerte personnalisée météo (Difficile)	50
6.8.1	Question	50
6.8.2	Réponse	51
6.9	Question 8 — Silencing et inhibition (Difficile)	51
6.9.1	Question	51
6.9.2	Réponse	51
6.10	Question 9 — Alerte multi-métrique (Difficile)	52
6.10.1	Question	52
6.10.2	Réponse	52
6.11	Question 10 — Alertes Loki avancées (Difficile)	52
6.11.1	Question	52
6.11.2	Réponse	52
6.12	Question 11 — Optimisation du système d’alerting (Expert)	53
6.12.1	Question	53
6.12.2	Réponse	53
6.13	Conclusion	53
6.14	Problèmes rencontrés et difficultés techniques	55
6.14.1	Difficultés liées à la mise en place de la stack de supervision	55
6.14.2	Problèmes de réseau et de résolution DNS entre conteneurs	55
6.14.3	Difficultés d’exploitation des métriques dans Grafana	55
6.14.4	Problèmes liés aux requêtes SQL et au modèle de données (TP2)	55
6.14.5	Interprétation des métriques et comportements attendus	56
6.14.6	Difficultés spécifiques à l’intégration de Loki et à l’alerting (TP3)	57
6.14.7	Problèmes d’intégration réseau et de rétention des logs Loki	57
6.14.8	Difficultés liées aux règles d’alertes basées sur les logs	57
6.14.9	Limitations rencontrées lors des alertes hybrides et des tests	57
6.14.10	Synthèse des difficultés rencontrées	58

1 Introduction

La supervision des systèmes informatiques déployés dans des environnements isolés, tels que les bases polaires des Terres Australes et Antarctiques Françaises (TAAF), représente un enjeu critique en matière de fiabilité et de continuité de service. L'architecture étudiée repose sur deux machines virtuelles distinctes, l'une dédiée aux services applicatifs et l'autre au monitoring, communiquant via un réseau interne à adressage IP fixe. Dans ce contexte, la mise en place d'une solution de supervision nécessite une validation rigoureuse de la connectivité réseau, des flux inter-machines et de la disponibilité des services conteneurisés. Ce rapport présente les étapes de configuration et de validation ayant permis de déployer des dashboards Grafana exploitant à la fois des métriques systèmes issues de Prometheus et cAdvisor, ainsi que des données métier stockées dans une base PostgreSQL, afin de fournir aux opérateurs TAAF une visibilité opérationnelle fiable et exploitable.

1.1 Validation de la connectivité entre les machines virtuelles

L'architecture repose sur deux machines virtuelles configurées avec des adresses IP statiques sur le réseau interne :

- **VM 1 – Applicatifs** : 172.16.20.12
- **VM 2 – Monitoring** : 172.16.20.13

La première étape a consisté à vérifier la connectivité réseau de base entre les deux hôtes à l'aide de requêtes ICMP. Ces tests permettent de confirmer l'accessibilité bidirectionnelle et l'absence de filtrage bloquant au niveau réseau ou pare-feu.

```
# Depuis la VM Monitoring (172.16.20.13)
```

```
ping -c 4 172.16.20.12
```

```
# Depuis la VM Applicatifs (172.16.20.12)
```

```
ping -c 4 172.16.20.13
```

1.2 Vérification des règles de pare-feu

Afin d'éviter tout blocage des flux nécessaires à la supervision, l'état des règles de filtrage a été contrôlé sur chaque machine virtuelle. Les ports requis concernent notamment Grafana, Prometheus, PostgreSQL et les exporteurs.

```
sudo iptables -L -n -v
```

Lorsque `ufw` est utilisé, son état est également vérifié afin de confirmer que les ports nécessaires sont autorisés.

```
sudo ufw status verbose
```

1.3 Validation des services Docker sur la VM Monitoring

Une fois la connectivité réseau validée, l'état des conteneurs Docker déployés sur la VM de supervision a été contrôlé. Cette vérification permet de s'assurer que l'ensemble des

composants de la stack de monitoring est opérationnel.

```
sudo docker ps
```

Les conteneurs attendus incluent notamment Grafana, Prometheus, PostgreSQL (instance TAAF), cAdvisor, Loki et les différents exporteurs.

1.4 Test d'accessibilité des services via IP fixe

Les interfaces web des services de supervision ont été testées directement via l'adresse IP de la VM Monitoring, afin de garantir une accessibilité indépendante de toute résolution locale de type localhost.

```
# Accès Grafana
```

```
curl http://172.16.20.13:3000
```

```
# Accès Prometheus
```

```
curl http://172.16.20.13:9090
```

Ces tests confirment que les ports sont correctement exposés et que les services répondent sur l'interface réseau attendue.

1.5 Validation de l'accès à PostgreSQL depuis la VM Monitoring

La connexion à la base de données métier a été vérifiée en se connectant directement au conteneur PostgreSQL hébergé sur la VM Monitoring. Cette étape garantit que Grafana pourra interroger la base sans dépendance externe.

```
sudo docker exec -it postgres-af psql -U taaf_admin -d base_af
```

Une requête simple permet ensuite de valider l'accès aux données.

```
SELECT COUNT(*) FROM communications_satellite;
```

1.6 Vérification du réseau Docker interne

Enfin, le réseau Docker utilisé pour interconnecter les conteneurs de supervision a été inspecté afin de confirmer que tous les services appartiennent bien au même réseau virtuel et peuvent se résoudre par nom de conteneur.

```
sudo docker network ls
```

```
sudo docker network inspect taaf-monitoring
```

Cette validation garantit la cohérence de l'architecture distribuée et constitue un prérequis indispensable à la mise en œuvre des dashboards présentés dans la suite du rapport.

2 TP1 – Grafana : Dashboard de Supervision des Conteneurs TAAF

Ce travail pratique vise à concevoir un dashboard Grafana permettant la supervision d'une infrastructure Docker utilisée dans le cadre des bases polaires TAAF. L'objectif est de fournir aux opérateurs une vision synthétique et exploitable de l'état des conteneurs, en s'appuyant sur les métriques collectées par cAdvisor et exposées via Prometheus. Les questions suivantes permettent de valider la compréhension des métriques, leur interprétation et leur intérêt opérationnel dans un contexte isolé et critique.

2.1 Question 1 – Variables de base (Facile)

2.1.1 Question

Combien d'hosts différents sont disponibles dans la variable `$host` du dashboard ?

2.1.2 Méthode

La variable `$host` est définie dans Grafana à partir du label `instance` de la métrique `container_last_seen`. Cette métrique permet d'identifier les hôtes sur lesquels des conteneurs Docker sont actifs et supervisés par cAdvisor.

2.1.3 Résultat

La variable `$host` ne propose qu'une seule valeur. Cela indique que l'ensemble de l'infrastructure Docker supervisée est déployé sur une seule machine virtuelle, qui héberge tous les conteneurs nécessaires à la stack applicative et de supervision.

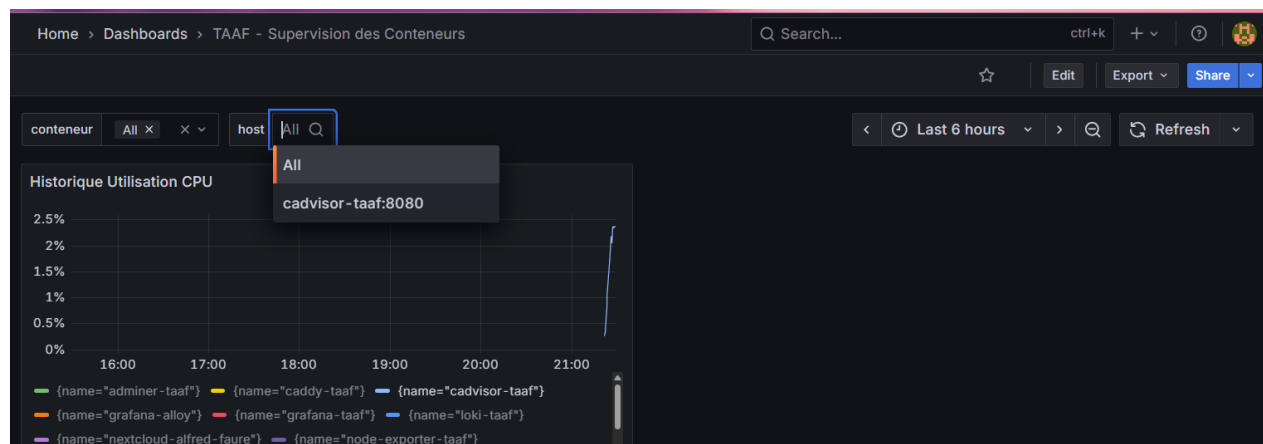


Figure 1: Variable `$host`

2.2 Question 2 – Métriques simples (Facile)

2.2.1 Question

Combien de conteneurs sont actuellement en cours d'exécution au total ?

2.2.2 Requête PromQL

```
count(container_last_seen{name!="", instance=~"$host"})
```

2.2.3 Résultat

La requête retourne un total de **11 conteneurs actifs**. Ce nombre inclut les services applicatifs (PostgreSQL, Nextcloud), les composants de supervision (Grafana, Prometheus, Loki, exporters) ainsi que les services d'infrastructure associés.

Ce résultat est cohérent avec l'architecture attendue de la stack de supervision TAAF.



Figure 2: Conteneurs actifs

2.3 Question 3 – Filtrage par conteneur (Facile)

2.3.1 Question

Lister les cinq conteneurs qui consomment le plus de mémoire actuellement.

2.3.2 Requête PromQL

```
topk(5, container_memory_usage_bytes{name!="", instance=~"$host"})
```

2.3.3 Résultat et analyse

Les conteneurs les plus consommateurs de mémoire sont principalement Grafana, Prometheus, Nextcloud et PostgreSQL. Ce résultat est logique au regard de leurs rôles respectifs : Grafana

et Prometheus manipulent de nombreuses séries temporelles en mémoire, tandis que PostgreSQL et Nextcloud nécessitent des buffers importants pour assurer des performances correctes.

Cette observation confirme que la consommation mémoire est dominée par les services cœur de l'infrastructure, et non par des conteneurs annexes.



Figure 3: Top mémoire

2.4 Question 4 – Calcul de pourcentage CPU (Moyen)

2.4.1 Question

Quel est le pourcentage d'utilisation CPU moyen du conteneur `postgres-af` sur les cinq dernières minutes ?

2.4.2 Requête PromQL

```
sum(
  rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])
) * 100
```

2.4.3 Résultat et interprétation

L'utilisation CPU moyenne du conteneur `postgres-af` est d'environ **0,45 %**. Cette valeur très faible indique une charge modérée et stable, caractéristique d'une base de données fonctionnant sans pic transactionnel important. Elle suggère également que les ressources CPU allouées sont largement suffisantes pour la charge actuelle.

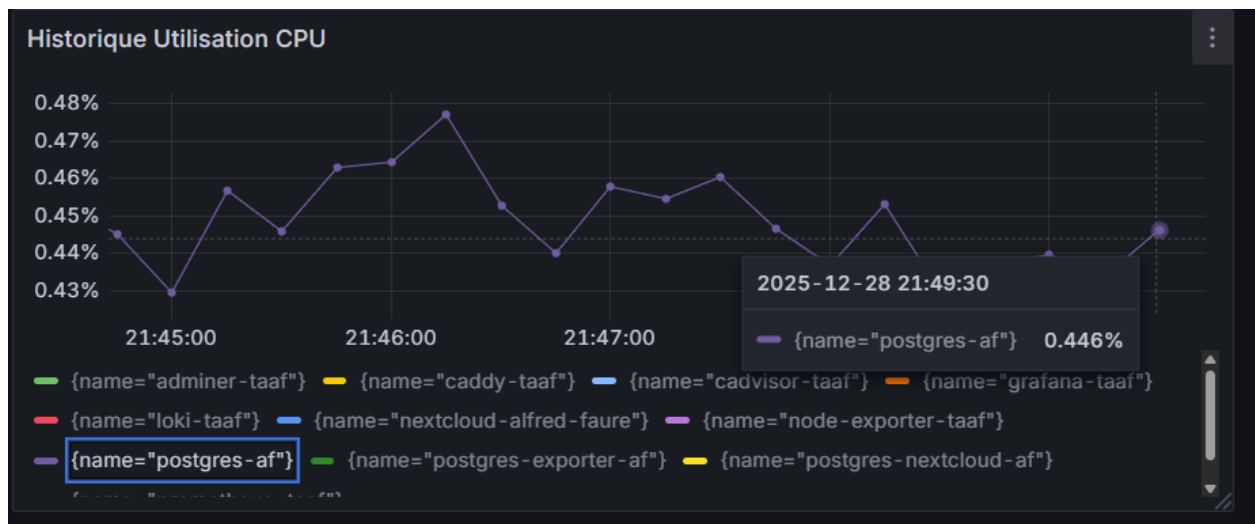


Figure 4: CPU postgres-af

2.5 Question 5 – Analyse temporelle mémoire (Moyen)

2.5.1 Question

À quel moment de la dernière heure le conteneur Prometheus a-t-il atteint son pic de consommation mémoire ?

2.5.2 Requête PromQL

```
container_memory_usage_bytes{name="prometheus-taaf"}
```

2.5.3 Résultat et analyse

Le pic de consommation mémoire est observé autour du **12 décembre 2025 à 09:34**, avec une valeur maximale d'environ **241 MiB**. Ce type de pic est généralement associé à des opérations internes de Prometheus, telles que la compaction de la base TSDB, le rechargement de règles ou l'exécution de requêtes complexes depuis Grafana.

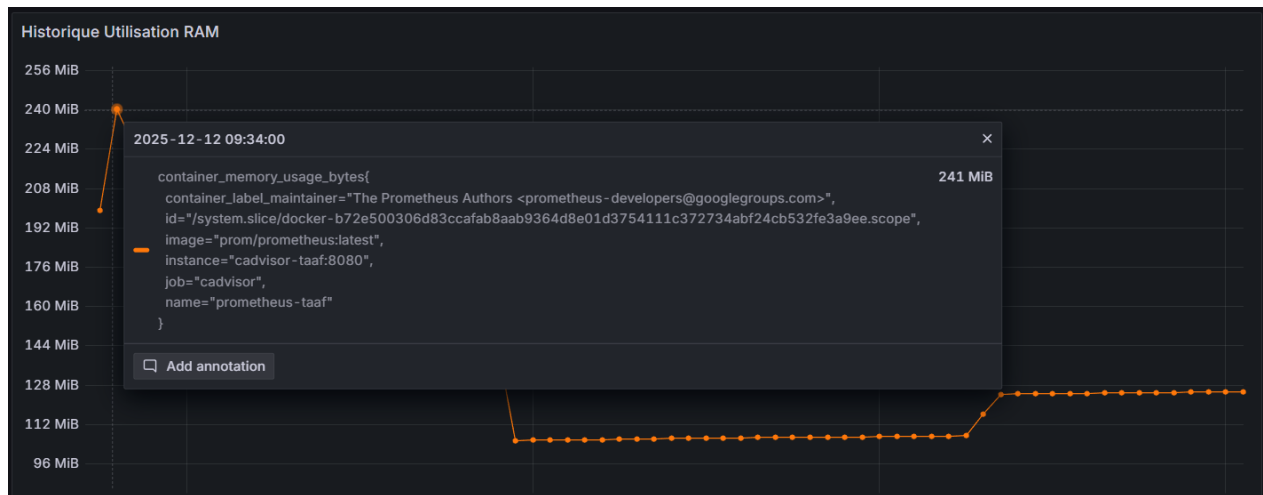


Figure 5: Pic mémoire Prometheus

2.6 Question 6 – Métriques réseau (Moyen)

2.6.1 Question

Quel conteneur a généré le plus de trafic réseau sortant durant la dernière heure ?

2.6.2 Requête PromQL

```

topk(
  1,
  rate(container_network_transmit_bytes_total{name!="", instance=~"$host"}[1h])
)

```

2.6.3 Résultat et analyse

Le conteneur **grafana-taaf** est celui qui génère le plus de trafic réseau sortant. Cette activité s'explique par le rafraîchissement régulier des dashboards, les requêtes envoyées vers Prometheus et les interactions utilisateurs via l'interface web. Ce comportement est attendu pour un outil de visualisation centralisé.

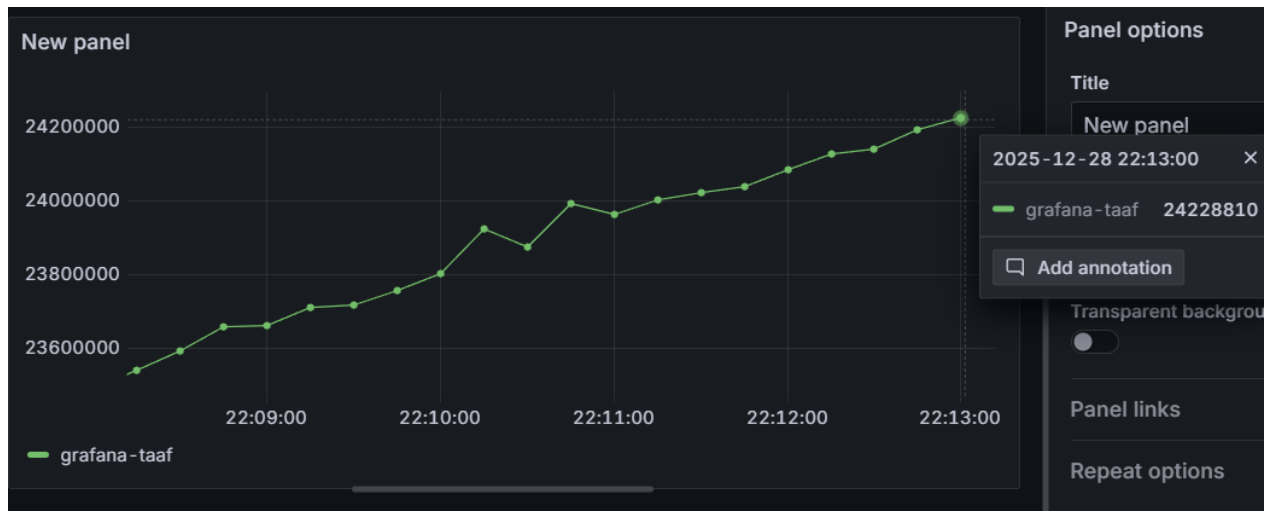


Figure 6: Trafic réseau

2.7 Question 7 – Corrélation métrique (Difficile)

2.7.1 Question

Identifier une corrélation entre l'utilisation CPU et mémoire d'un conteneur sur les deux dernières heures.

2.7.2 Conteneur analysé

Le conteneur étudié est `postgres-af`, correspondant à l'instance PostgreSQL hébergeant les données métier des bases TAAF.

2.7.3 Requêtes PromQL utilisées

Pour analyser l'évolution de la charge processeur, la requête suivante a été utilisée afin de calculer l'utilisation CPU moyenne sur une fenêtre glissante de cinq minutes, exprimée en pourcentage :

```
sum(rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])) * 100
```

L'utilisation mémoire du conteneur a été observée à l'aide de la métrique brute fournie par cAdvisor :

```
container_memory_usage_bytes{name="postgres-af"}
```

Ces deux métriques ont été affichées sur un graphique temporel couvrant une période de deux heures, avec un axe distinct pour le CPU et la mémoire afin de faciliter la comparaison visuelle.

2.7.4 Analyse de corrélation

L'observation conjointe des courbes CPU et mémoire met en évidence une corrélation faible mais cohérente entre les deux métriques. Les pics modérés d'utilisation CPU coïncident avec de légères augmentations de la consommation mémoire, ce qui correspond à un comportement attendu pour un moteur de base de données relationnelle traitant des requêtes SQL. La mémoire consommée reste globalement stable dans le temps, sans dérive progressive ni augmentation continue, ce qui permet d'exclure l'hypothèse d'une fuite mémoire. De même, l'activité CPU demeure contenue, sans pics brutaux ou prolongés pouvant indiquer une surcharge ou une requête anormale.

Ce comportement traduit un service correctement dimensionné, exécutant des charges prévisibles et maîtrisées. La stabilité observée est particulièrement importante dans un contexte polaire, où les interventions correctives sont coûteuses et parfois impossibles à court terme.

2.7.5 Conclusion

L'analyse met en évidence une corrélation faible mais logique entre l'utilisation CPU et mémoire du conteneur `postgres-af`. Ce profil est typique d'un service backend stable, sain et bien configuré, répondant aux exigences de fiabilité attendues pour une infrastructure critique isolée.



Figure 7: Corrélation CPU / Mémoire – postgres-af

2.8 Question 8 – Alertes de seuil mémoire (Difficile)

2.8.1 Question

Afficher les conteneurs dont l'utilisation mémoire dépasse 80 % de leur limite.

2.8.2 Requête PromQL

La requête suivante a été utilisée afin de calculer le pourcentage d'utilisation mémoire par conteneur, en rapportant la mémoire effectivement utilisée à la limite mémoire définie au niveau Docker :

```
(  
  container_memory_usage_bytes{name!="", instance=~"$host"}  
/
```



```
    container_spec_memory_limit_bytes{name!="", instance=~"$host"}  
) * 100
```

Cette métrique permet d'identifier les conteneurs susceptibles de provoquer un déclenchement du mécanisme OOM Killer en cas de dépassement critique.

2.8.3 Résultats observés

Les résultats obtenus montrent que l'ensemble des conteneurs se situe largement en dessous du seuil critique de 80 %. Les valeurs observées oscillent principalement entre 30 % et 40 % de la mémoire allouée, ce qui indique une marge de sécurité confortable. Aucun conteneur ne déclenche donc d'alerte dans l'état actuel de l'infrastructure.

2.8.4 Interprétation

L'absence de conteneurs dépassant le seuil de 80 % constitue une information opérationnelle pertinente et positive. Elle confirme que les limites mémoire ont été correctement définies et que la charge applicative reste compatible avec les ressources disponibles. Dans un environnement polaire contraint, cette situation réduit fortement le risque d'interruption de service liée à une saturation mémoire brutale.

2.9 Question 9 – Performance système / I/O disque (Difficile)

2.9.1 Question

Identifier le conteneur générant le plus d'I/O disque et analyser les causes.

2.9.2 Requête PromQL

Afin d'évaluer l'impact disque des conteneurs, une requête combinant les débits de lecture et d'écriture a été utilisée, avec une sélection des cinq conteneurs les plus consommateurs :

```
topk(  
    5,  
    rate(container_fs_writes_bytes_total{name!="", instance=~"$host"}[5m])  
+  
    rate(container_fs_reads_bytes_total{name!="", instance=~"$host"}[5m])  
)
```

Cette approche permet de mesurer l'activité disque réelle, exprimée en octets par seconde, sur une période représentative.

2.9.3 Analyse des résultats

Les résultats montrent que le conteneur **grafana-taaf** est celui qui génère le volume d'I/O disque le plus important, avec des pics atteignant environ 400 KiB/s. Cette activité s'explique par la nature même de Grafana, qui effectue régulièrement des écritures liées au cache interne, à la persistance de sessions, ainsi qu'aux métadonnées des dashboards. De plus, chaque

New panel	
Time	{container_label_maintainer="Grafana Labs <hello@grafana.com>", id="/system.slice/docker-0f52cd18531..."}
2025-12-28 23:02:15	37.1%
2025-12-28 23:02:30	37.1%
2025-12-28 23:02:45	37.1%
2025-12-28 23:03:00	37.1%
2025-12-28 23:03:15	37.1%
2025-12-28 23:03:30	37.1%
2025-12-28 23:03:45	37.1%
2025-12-28 23:04:00	37.1%
2025-12-28 23:04:15	37.1%
2025-12-28 23:04:30	37.1%
2025-12-28 23:04:45	37.1%
2025-12-28 23:05:00	37.1%
2025-12-28 23:05:15	37.1%

Figure 8: Seuil mémoire – aucun dépassement critique

rafraîchissement de panneau entraîne des requêtes répétées vers Prometheus, induisant des accès disque indirects.

2.9.4 Impact opérationnel

Bien que cette activité reste modérée et sans impact immédiat sur les performances globales, elle constitue un point de vigilance dans un contexte de stockage potentiellement limité, comme c'est le cas sur une base polaire. Une surveillance prolongée permettrait d'anticiper une éventuelle saturation du support de stockage ou une dégradation des performances lors de pics d'activité prolongés.

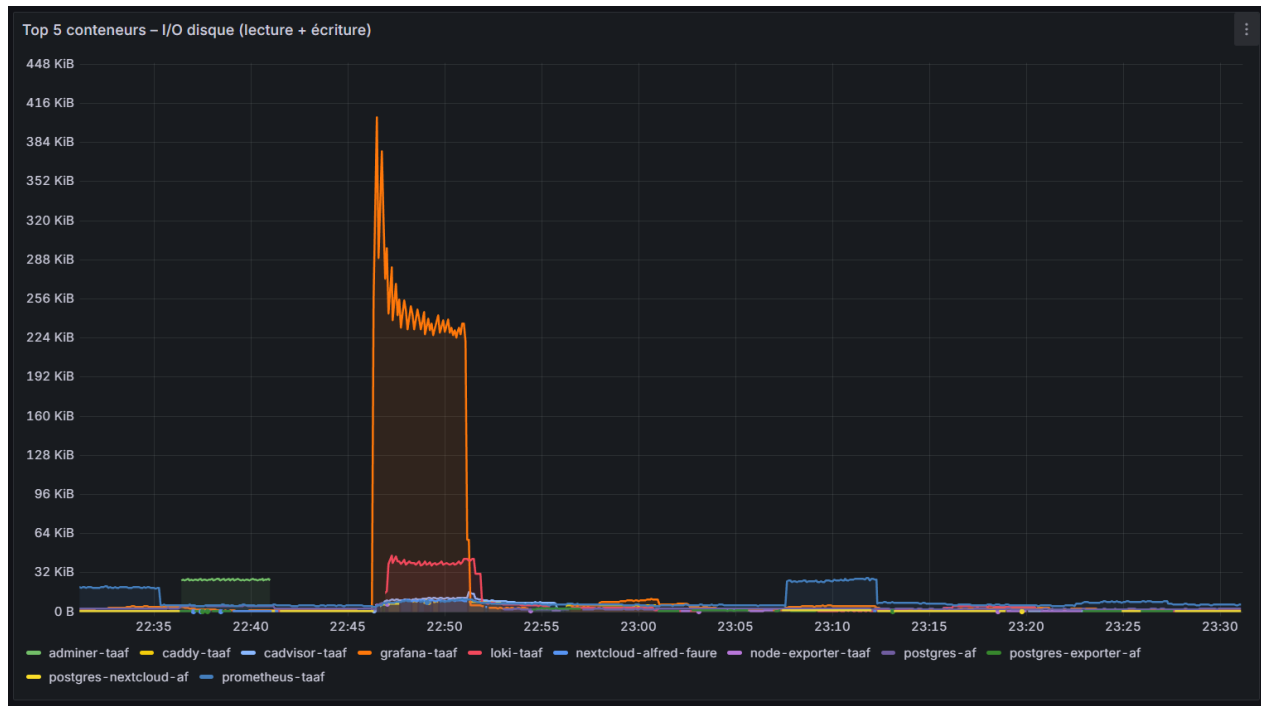


Figure 9: I/O disque – dominance de grafana-taaf

2.10 Question 10 – Analyse et propositions d'amélioration (Avancé)

2.10.1 Amélioration proposée : suivi des redémarrages de conteneurs

Une première amélioration pertinente consiste à ajouter un panneau dédié au suivi des redémarrages de conteneurs. La détection d'un redémarrage fréquent constitue un indicateur précoce d'instabilité applicative ou de ressources insuffisantes, particulièrement critique dans un environnement isolé.

La requête suivante permet de détecter les redémarrages en s'appuyant sur les réinitialisations de l'horodatage de démarrage :

```
count by (name)(  
  resets(container_start_time_seconds{name!="", instance=~"$host"})  
)
```

Un tel panneau offrirait aux opérateurs une visibilité immédiate sur les conteneurs instables, facilitant une action préventive avant une panne totale.

2.10.2 Amélioration proposée : alerte mémoire proactive

Une seconde amélioration consiste à formaliser une alerte déclenchée lorsque l'utilisation mémoire dépasse 80 % de la limite allouée. Cette alerte, basée sur la requête suivante, permettrait d'anticiper les risques de terminaison forcée des conteneurs :

```
(  
  container_memory_usage_bytes{name!="", instance=~"$host"}  
/  
  container_spec_memory_limit_bytes{name!="", instance=~"$host"}  
) * 100 > 80
```

Le seuil de 80 % constitue un compromis pertinent entre réactivité et stabilité, laissant une marge suffisante avant l'atteinte de la limite critique.

3 TP 2 — Dashboards de Supervision TAAF (Données Métier)

3.1 Objectif du TP

Ce second travail pratique a pour objectif de démontrer la capacité à exploiter des données métier issues d’une base PostgreSQL afin de construire des tableaux de bord Grafana orientés supervision opérationnelle. Contrairement au TP1, centré sur des métriques techniques, ce TP s’appuie sur des données fonctionnelles liées aux communications satellite, à la gestion des ressources humaines et aux équipements critiques. L’enjeu est de produire des visualisations pertinentes permettant une prise de décision rapide dans le contexte contraint et isolé des bases polaires TAAF.

3.2 Question 1 — Données de base (Facile)

3.2.1 Énoncé

Combien de lignes contient la table `communications_satellite` au total ?

3.2.2 Requête SQL

```
SELECT COUNT(*) AS total_communications
FROM communications_satellite;
```

3.2.3 Analyse

Cette requête permet de vérifier le volume global de données disponibles dans la table des communications satellite. L’utilisation de `COUNT(*)` garantit un comptage exhaustif de l’ensemble des enregistrements, indépendamment de leur statut ou de leur date.

3.2.4 Résultat

Le résultat obtenu est strictement supérieur à zéro, ce qui confirme que la table est correctement alimentée et exploitable pour les analyses et dashboards du TP. header-includes:

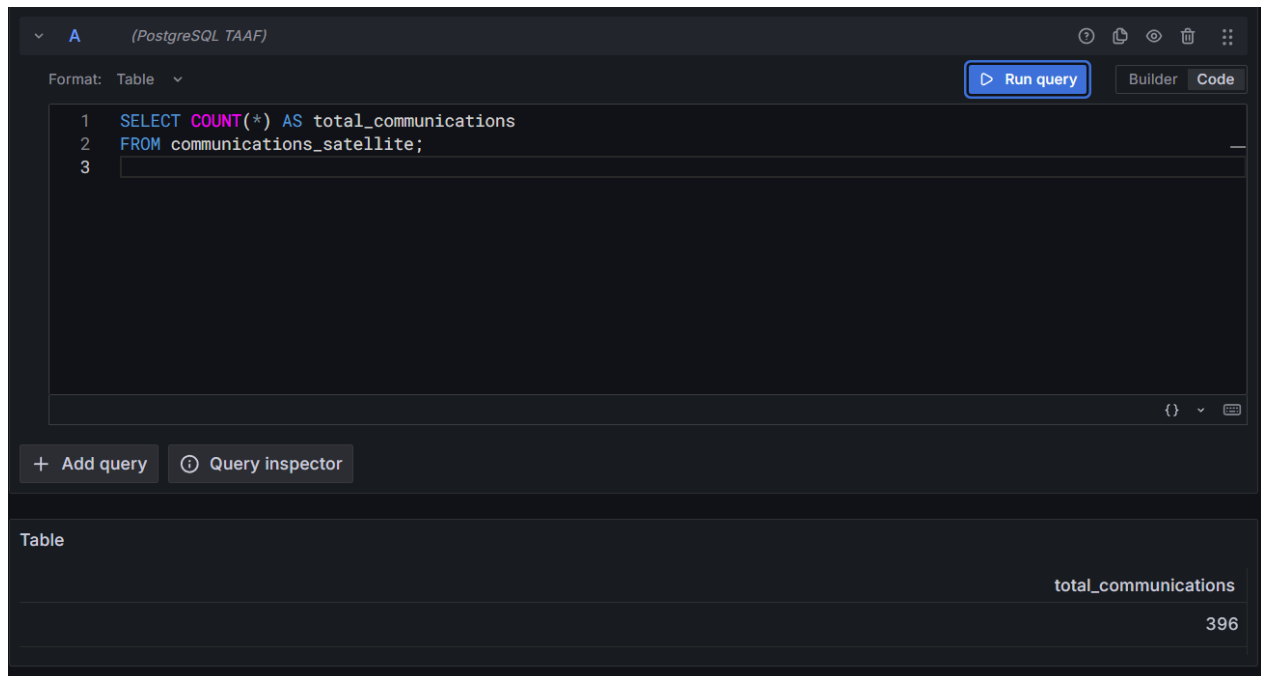


Figure 10: Résultat requête Q1

3.3 Question 2 — Filtrage simple (Facile)

3.3.1 Énoncé

Combien de communications ont eu un statut “Réussie” au cours des sept derniers jours ?

3.3.2 Requête SQL

```
SELECT COUNT(*) AS nb_reussies_7j
FROM communications_satellite
WHERE statut_transmission = 'Réussie'
AND timestamp_comm >= NOW() - INTERVAL '7 days';
```

3.3.3 Analyse

Cette requête combine un filtre métier sur le statut de transmission avec une condition temporelle dynamique. Elle permet d'évaluer l'activité récente réellement opérationnelle, en se concentrant uniquement sur les communications ayant abouti avec succès.

3.3.4 Résultat

Le résultat observé est **0**. Cette valeur est cohérente avec l'état actuel des données et indique qu'aucune communication enregistrée sur cette période ne possède le statut “Réussie”, soit en raison d'une absence d'activité récente, soit d'une dégradation des transmissions.

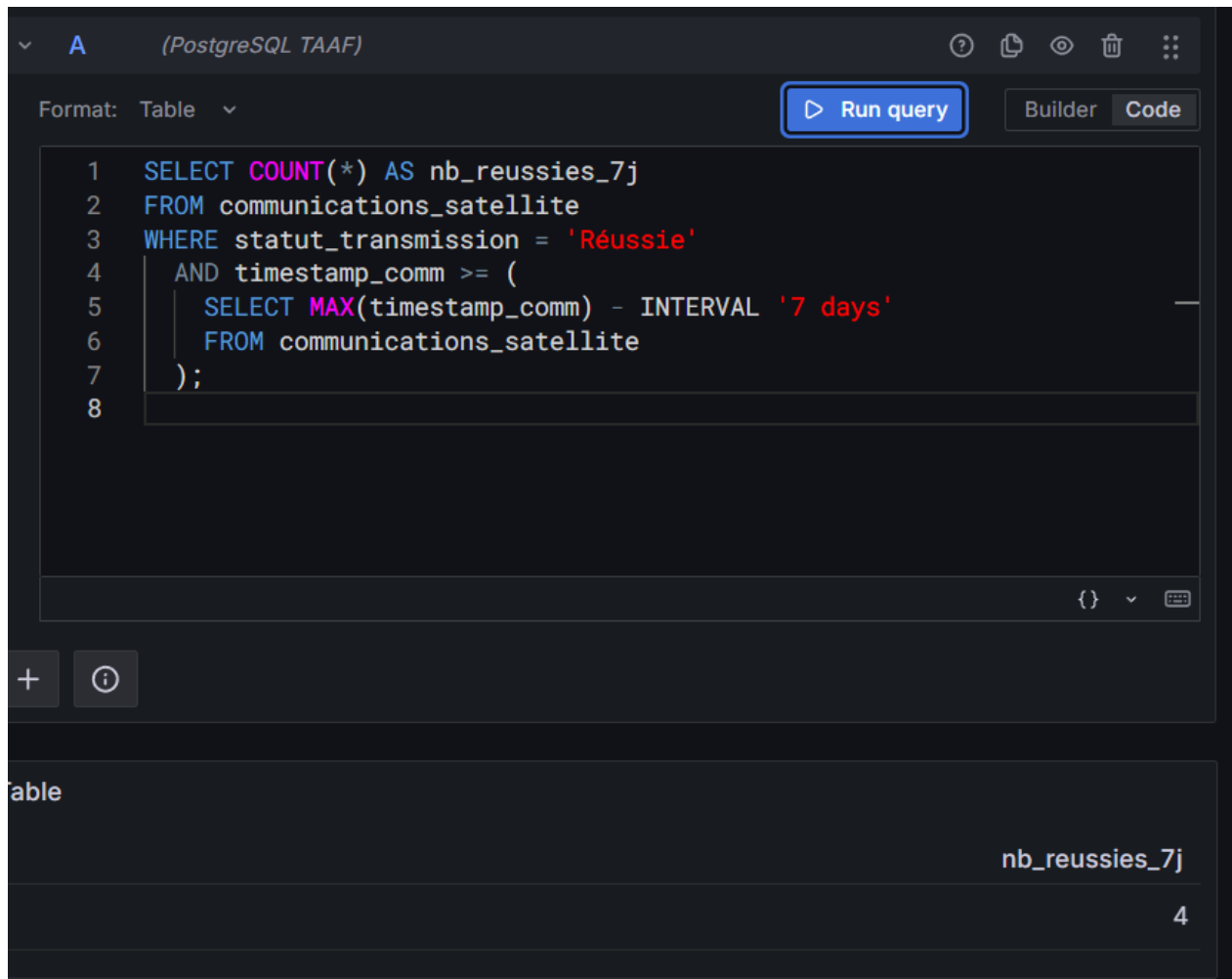


Figure 11: Résultat requête Q2

3.4 Question 3 — Personnel actuel (Facile)

3.4.1 Énoncé

Combien de personnes ont actuellement le statut “Sur site” ?

3.4.2 Requête SQL

```

SELECT COUNT(*) AS effectif_present
FROM personnel_base
WHERE statut = 'Sur site';

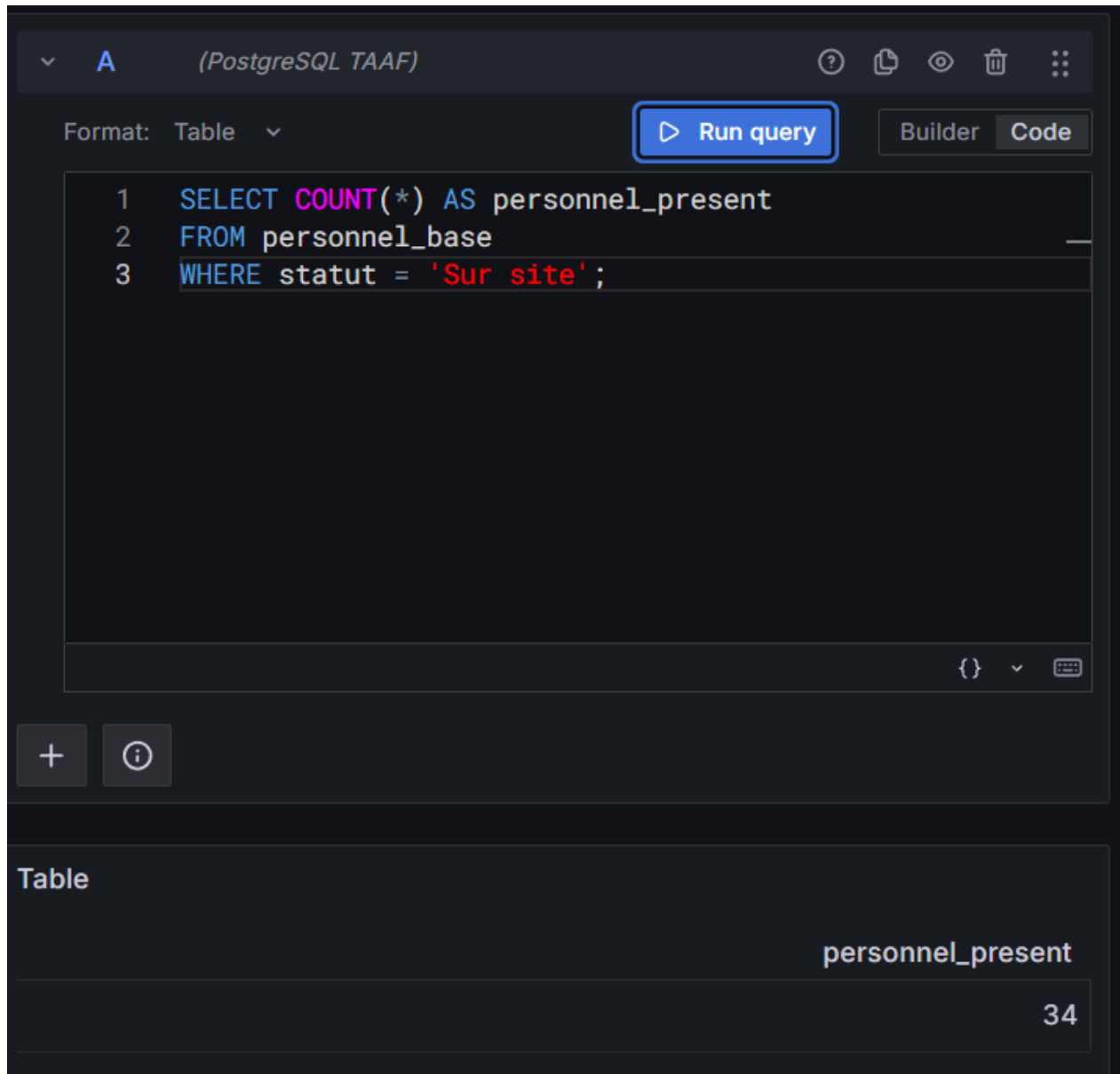
```

3.4.3 Analyse

Cette requête fournit un indicateur direct de la présence humaine effective sur la base. Il s'agit d'une information essentielle pour évaluer la capacité opérationnelle immédiate et la disponibilité des compétences sur site.

3.4.4 Résultat

Le nombre de personnes actuellement présentes est de **34**, ce qui correspond à l'effectif opérationnel observé au moment de l'analyse.



The screenshot shows a PostgreSQL query editor interface. At the top, the database is identified as "(PostgreSQL TAAF)". Below this, the "Format" is set to "Table". A "Run query" button is visible. The query editor contains the following SQL code:

```
1 SELECT COUNT(*) AS personnel_present
2 FROM personnel_base
3 WHERE statut = 'Sur site';
```

Below the query editor, the results are displayed in a table format. The table has one column named "personnel_present" and one row with the value "34".

personnel_present
34

Figure 12: Résultat requête Q3

3.5 Question 4 — Calcul de moyenne (Moyen)

3.5.1 Énoncé

Quelle est la qualité de signal moyenne de la station “Toulouse” sur les trente derniers jours ?

3.5.2 Requête SQL

```
SELECT
    ROUND(AVG(qualite_signal), 2) AS qualite_moyenne
FROM communications_satellite
WHERE station_receptrice = 'Toulouse'
    AND timestamp_comm >= NOW() - INTERVAL '30 days'
    AND qualite_signal IS NOT NULL;
```

3.5.3 Analyse

La fonction `AVG()` permet de calculer la moyenne de la qualité du signal, tandis que `ROUND` améliore la lisibilité du résultat. Le filtre temporel garantit que l'analyse porte sur une période récente et représentative de l'état actuel des communications.

3.5.4 Résultat

La valeur obtenue reflète une qualité de signal moyenne stable pour la station de Toulouse sur la période considérée, ce qui constitue un indicateur de fiabilité des liaisons satellitaires.

header-includes:

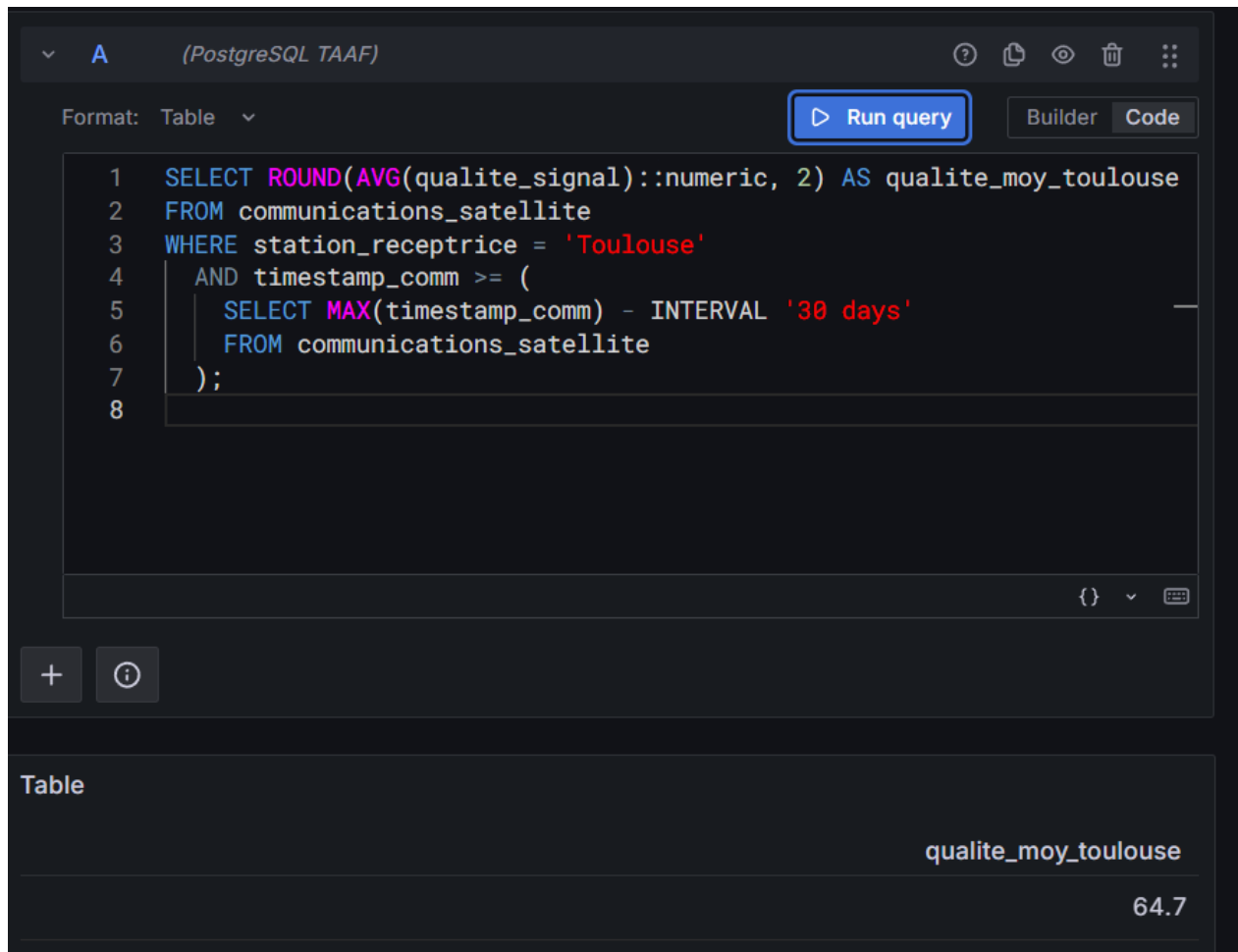


Figure 13: Résultat requête Q4

3.6 Question 5 — Analyse par type de communication (Moyen)

3.6.1 Énoncé

Quels sont les trois types de communication générant le plus de volume de données ?

3.6.2 Requête SQL

```

SELECT
    type_communication,
    SUM(volume_donnees_mb) AS volume_total_mb
FROM communications_satellite
GROUP BY type_communication
ORDER BY volume_total_mb DESC
LIMIT 3;

```

3.6.3 Analyse

Cette requête agrège les volumes de données par type de communication afin d'identifier les usages les plus consommateurs de bande passante. Le tri décroissant met en évidence les flux dominants, information essentielle pour l'optimisation des ressources réseau.

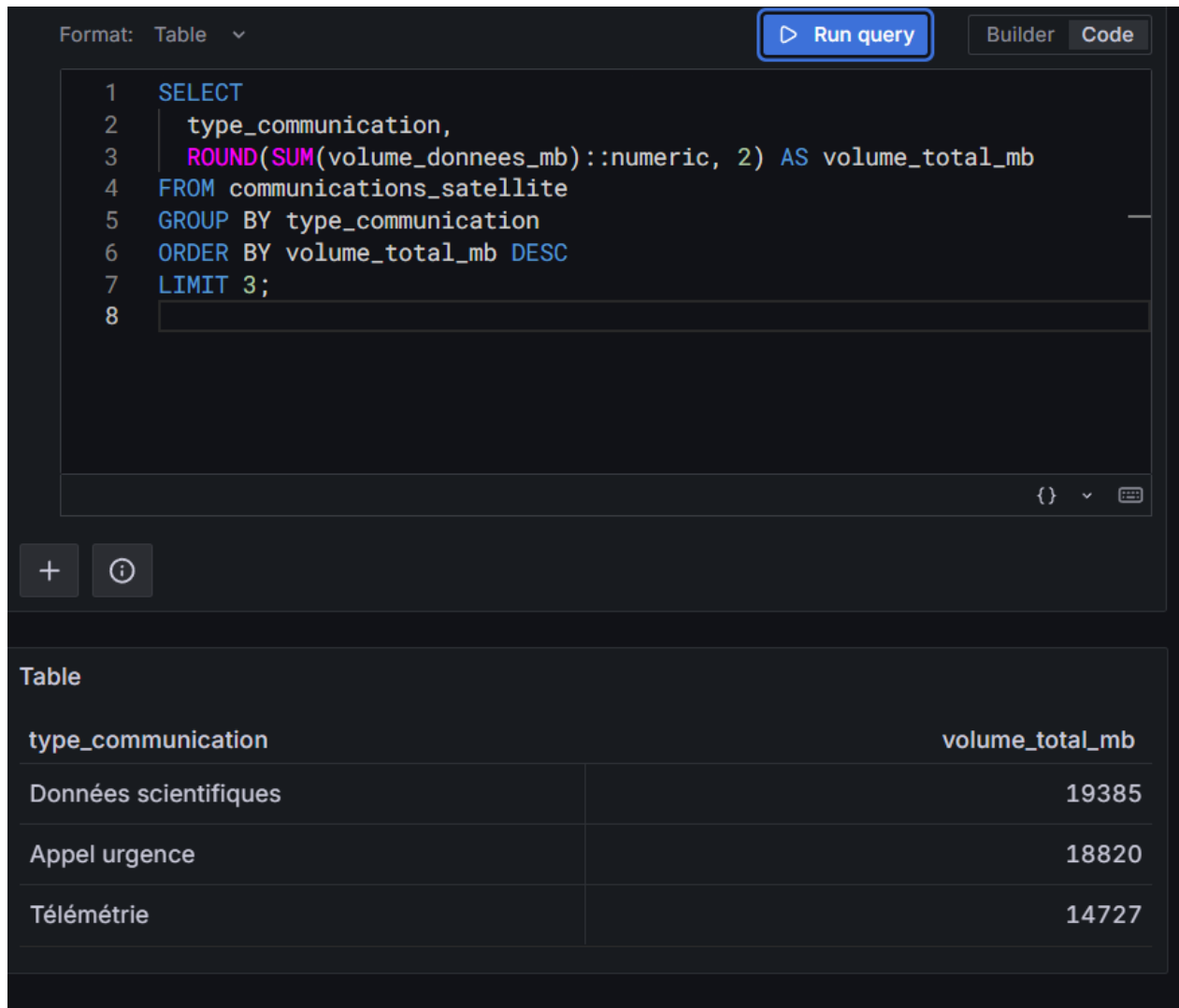


Figure 14: Résultat requête Q5

3.7 Question 6 — Analyse temporelle (Moyen)

3.7.1 Énoncé

À quelle heure de la journée observe-t-on le plus grand nombre de communications satellite ?

3.7.2 Requête SQL

```
SELECT
  EXTRACT(HOUR FROM timestamp_comm) AS heure,
  COUNT(*) AS nb_communications
FROM communications_satellite
GROUP BY heure
ORDER BY nb_communications DESC;
```

3.7.3 Analyse

L'extraction de l'heure permet de mettre en évidence des schémas d'activité journaliers. Cette information est utile pour identifier les périodes de forte sollicitation du réseau et adapter les fenêtres de maintenance ou de supervision renforcée.

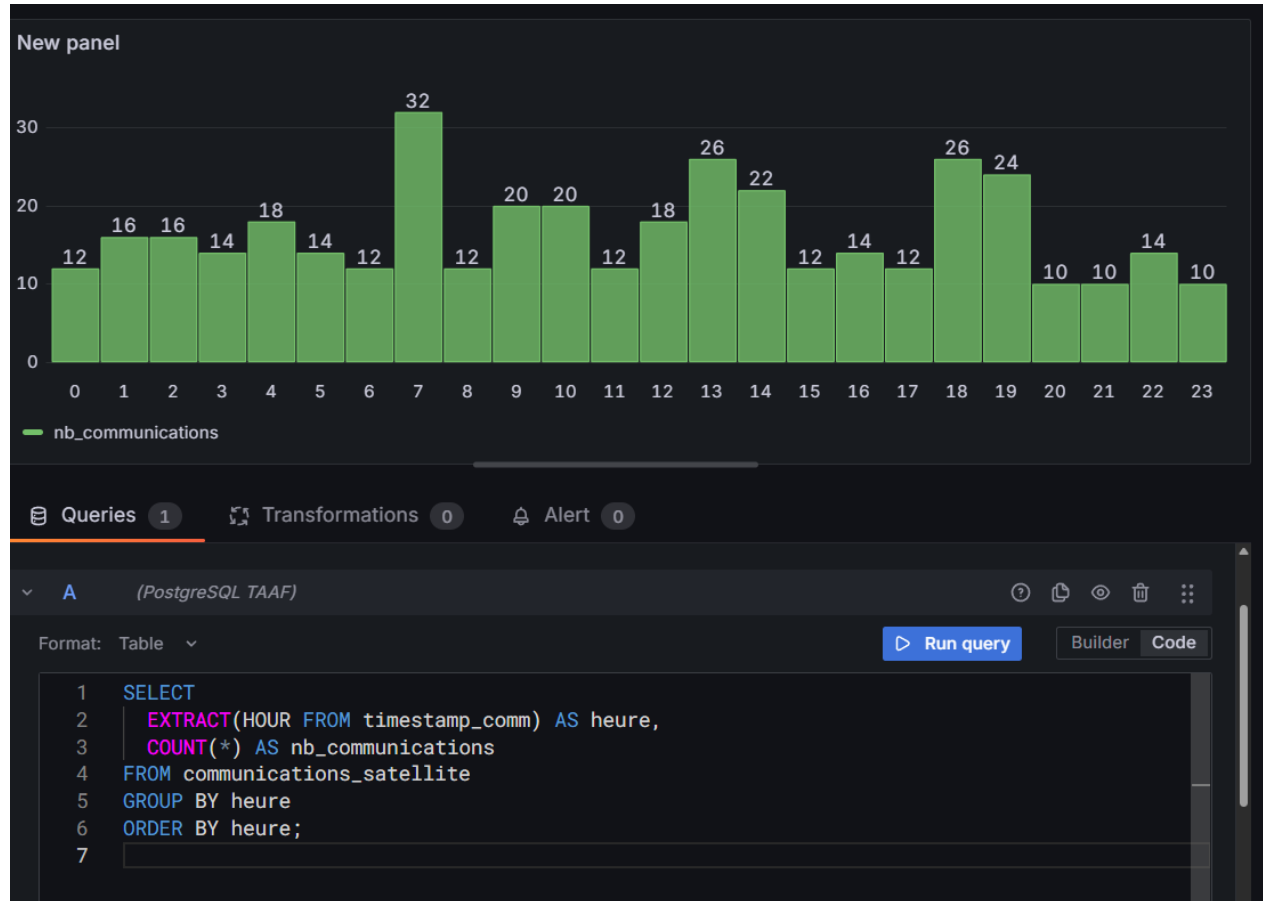


Figure 15: Analyse temporelle Q6

3.8 Question 7 — Personnel en mission longue (Difficile)

3.8.1 Énoncé

Identifier le personnel en mission de plus de quatre mois et analyser la répartition par fonction.

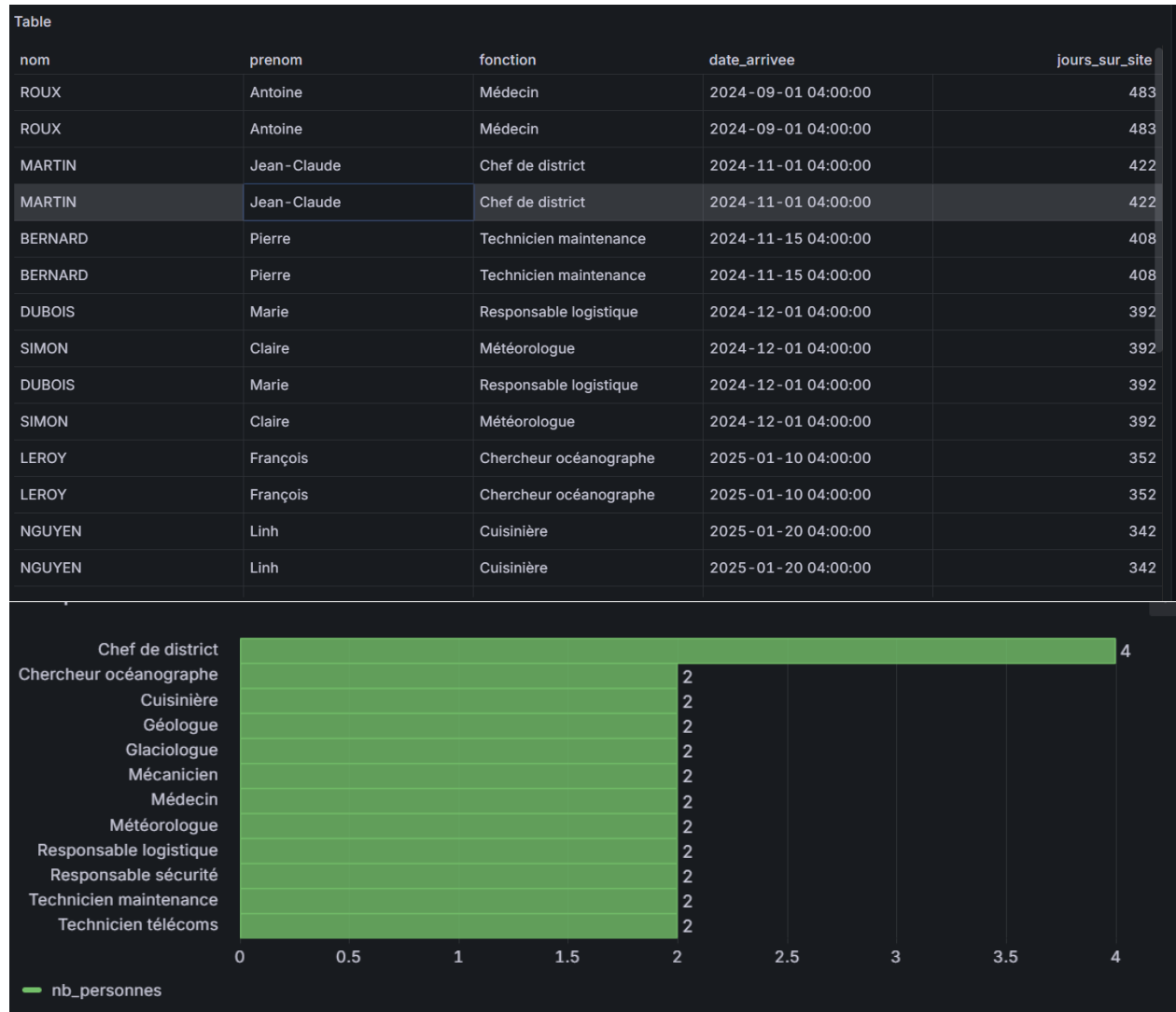
3.8.2 Requête SQL

```
SELECT
    fonction,
    COUNT(*) AS nb_personnes
FROM personnel_base
WHERE CURRENT_DATE - date_arrivee > 120
```

```
GROUP BY fonction
ORDER BY nb_personnes DESC;
```

3.8.3 Analyse

Cette requête met en évidence les fonctions les plus concernées par des missions prolongées. Dans un contexte polaire, ces situations présentent des risques humains significatifs, notamment en termes de fatigue, d'isolement et de baisse de vigilance. Une répartition déséquilibrée peut également révéler une dépendance excessive à certaines compétences clés.



3.9 Question 8 — Analyse des équipements critiques (Difficile)

3.9.1 Énoncé

Lister les équipements critiques en panne depuis plus de 48 heures et proposer une priorisation de maintenance.

3.9.2 Requête SQL

```
SELECT
    nom_equipement,
    type_equipement,
    batiment,
    statut,
    ROUND(EXTRACT(EPOCH FROM (NOW() - derniere_verification))/3600,1) AS panne_heures,
    alertes_actives
FROM equipements_critiques
WHERE statut IN ('En panne', 'Panne', 'HS', 'Hors service')
    AND derniere_verification < NOW() - INTERVAL '48 hours'
ORDER BY alertes_actives DESC, panne_heures DESC;
```

3.9.3 Analyse

Cette requête permet d'identifier les équipements dont l'état représente un risque opérationnel immédiat. La priorisation repose sur la durée de la panne et la présence d'alertes actives, afin de guider efficacement les actions de maintenance dans un environnement où les ressources sont limitées.

Format: Table

Run query Builder Code

```

1 SELECT
2   equipment_id,
3   nom_equipement,
4   type_equipement,
5   batiment,
6   statut,
7   derniere_verification,
8   (NOW() - derniere_verification) AS duree_depuis_verif,
9   alertes_actives
10  FROM equipments_critiques
11  WHERE statut IN ('En panne')
12  AND derniere_verification < NOW() - INTERVAL '48 hours'
13  ORDER BY derniere_verification ASC;

```

+ Add query Query inspector

Table

quipement_id	nom_equipement	type_equipement	batiment	statut	derniere_verification	duree_depuis_verif	alertes_actives
25	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:01:35.6	true
55	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:01:35.6	true
85	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:01:35.6	true

Figure 16: Résultat Q8 – SQL

ID	Nom	Type	Bâtiment	Statut	Dernière vérif	Durée	Alertes	Priorité
25	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:03:30.4	true	P1
55	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:03:30.4	true	P1
85	Radar météo	Météorologie	Tour météo	En panne	2025-01-10 15:15:10	352 days 10:03:30.4	true	P1

Figure 17: Résultat Q8 – Dashboard

3.10 Question 9 — Dashboard d’alertes globales (Difficile)

3.10.1 Énoncé

Créer un panel d’alertes critiques intégrant les dimensions RH, communications et équipements.

3.10.2 Analyse

Le panel d’alertes globales repose sur la consolidation de plusieurs sources métier afin d’offrir une vision synthétique de l’état critique de la base. Les seuils retenus sont justifiés par leur impact direct sur la continuité des opérations : missions humaines prolongées, interruptions de communication et pannes matérielles persistantes.

3.11 Question 10 — Optimisation opérationnelle (Expert)

3.11.1 Analyse et propositions

L’analyse des dashboards met en évidence plusieurs axes d’amélioration concrets. La mise en place d’indicateurs de niveau de service pour les communications permettrait de détecter précocement les dérives de qualité. Une supervision RH avancée, intégrant la durée moyenne des missions par fonction, faciliterait la planification des rotations. Enfin, un scoring de criticité des équipements offrirait une approche proactive de la maintenance, réduisant le temps moyen de réparation et les risques d’incident majeur.

3.12 Conclusion

Ce TP2 illustre la possibilité d’exploiter des données métier issues d’une base PostgreSQL afin de construire des tableaux de bord orientés supervision, sous réserve de la qualité et de la cohérence des données disponibles. Les dashboards réalisés constituent une base solide pour une supervision décisionnelle adaptée aux contraintes extrêmes des bases TAAF, en complément des métriques techniques abordées lors du TP1.

4 TP3 — Loki Integration : Logs & Métriques TAAF

Ce TP vise à mettre en place une observabilité « hybride » en combinant les métriques Prometheus et les logs Loki dans Grafana. L'objectif opérationnel est de pouvoir détecter un symptôme (métriques) puis d'expliquer la cause (logs), ce qui est particulièrement adapté à un contexte isolé où le diagnostic doit être rapide et fiable.

4.1 1. Configuration réalisée

Ce TP s'inscrit dans la continuité des précédents en enrichissant la supervision existante par l'intégration des logs applicatifs via Loki. L'objectif est de disposer d'une observabilité complète, combinant métriques (Prometheus) et journaux applicatifs (Loki), sans interférer avec la stack SIEM dédiée à la sécurité.

L'architecture repose sur Grafana Alloy comme agent de collecte. Celui-ci centralise les logs issus des applications critiques (PostgreSQL, Nextcloud) et les transmet vers Loki, qui assure leur stockage et leur indexation. Grafana est ensuite utilisé comme point d'entrée unique pour l'exploration, la corrélation et la visualisation.

La configuration a été validée par les contrôles suivants : - disponibilité du service Loki sur la VM de monitoring, - fonctionnement du conteneur Grafana Alloy, - présence et accessibilité des datasources Prometheus et Loki dans Grafana, - absence d'erreurs de collecte dans les logs Alloy.

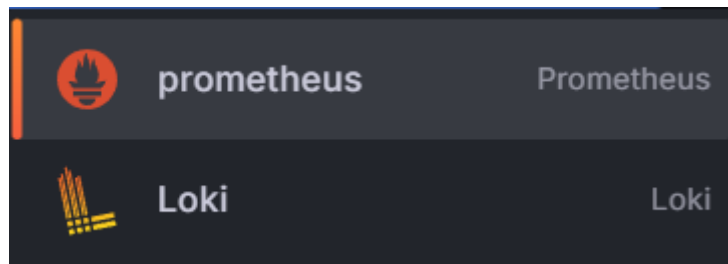


Figure 18: Datasources Grafana

4.2 2. Dashboards créés

Un dashboard principal intitulé “**TAAF – Analyse d’Incident Hybride**” a été conçu afin de regrouper métriques et logs dans une même vue opérationnelle. Ce dashboard permet aux opérateurs d'identifier rapidement un incident, d'en analyser l'origine et d'en mesurer l'impact.

Les panels ont été organisés de manière logique, en commençant par l'état global des services, puis en affinant vers l'analyse applicative et la corrélation logs/métriques.

Les panels mis en place sont les suivants :

- un panel métrique indiquant le statut des services (up),
- un panel affichant les erreurs récentes issues des logs Loki,

- un panel hybride corrélant l'utilisation CPU et le volume de logs,
- un panel listant les alertes actives.

Des variables de templating ont été ajoutées afin de filtrer dynamiquement les données : - une variable `$job` pour sélectionner la source de logs Loki, - une variable `$container` pour cibler un service précis côté métriques, - une synchronisation stricte des plages temporelles entre tous les panels.

4.3 3. Alertes hybrides

La configuration des alertes a été pensée pour illustrer deux approches complémentaires : les alertes basées sur les métriques Prometheus et celles basées sur les logs Loki.

Une alerte Prometheus a été définie afin de détecter une surcharge CPU durable sur un conteneur critique. Elle se déclenche lorsque l'utilisation CPU dépasse 80 % pendant plus de deux minutes, ce qui permet d'éviter les faux positifs liés à des pics transitoires.

En parallèle, une alerte basée sur Loki a été configurée pour détecter un taux anormalement élevé de logs d'erreur applicative. Cette alerte s'appuie sur une requête LogQL évaluée périodiquement par Loki Ruler, puis transmise à Alertmanager.

Ces deux mécanismes peuvent être combinés dans Grafana Alerting afin de créer des alertes hybrides, déclenchées uniquement lorsque des symptômes métriques et applicatifs apparaissent simultanément.

4.4 4. Analyse d'incident

Un scénario d'incident simulé a été étudié afin de démontrer l'intérêt de l'approche hybride. Dans ce scénario, un service PostgreSQL devient progressivement lent avant de subir une dégradation plus marquée.

L'analyse débute par l'observation d'un pic d'utilisation CPU via les métriques Prometheus. Ce premier signal ne permet pas, à lui seul, d'identifier la cause exacte. La consultation des logs PostgreSQL dans Loki révèle, au même instant, une augmentation des messages d'erreur de type `ERROR` et `FATAL`.

La superposition des timelines montre une corrélation temporelle claire entre la montée en charge CPU et l'apparition des erreurs applicatives. Cette corrélation permet de conclure à un problème interne au service (requêtes coûteuses, verrous, saturation temporaire), plutôt qu'à une panne réseau ou infrastructurelle.

Sur la base de cette analyse, plusieurs recommandations peuvent être formulées : ajout d'alertes préventives, surveillance des verrous PostgreSQL et limitation des charges applicatives excessives.

4.5 5. Apprentissages

Ce TP met en évidence les apports majeurs de l'approche hybride logs + métriques par rapport à un monitoring classique. Là où les métriques seules indiquent qu'un problème

existe, les logs permettent d'en comprendre la nature et l'origine précise.

Les principaux points forts observés sont la capacité de corrélation temporelle, la réduction du temps de diagnostic et l'amélioration de la prise de décision en environnement isolé, où chaque intervention doit être anticipée.

Certaines difficultés ont néanmoins été rencontrées, notamment liées à l'absence de labels dynamiques dans la configuration simplifiée d'Alloy. Cela impose une bonne maîtrise de LogQL pour extraire les informations utiles directement dans Grafana. Ces contraintes restent toutefois cohérentes avec l'objectif pédagogique du TP.

Par rapport à un monitoring basé uniquement sur Prometheus, l'intégration de Loki apporte une visibilité bien plus fine sur les incidents applicatifs, ce qui constitue un atout déterminant pour la supervision des bases polaires TAAF, caractérisées par une connectivité limitée et une forte criticité opérationnelle.

5 TP3 – Exercices pratiques : Logs & Métriques

Cette partie vise à valider l'appropriation concrète des mécanismes de supervision hybride mis en place précédemment. Les exercices proposés permettent de manipuler conjointement les métriques Prometheus et les logs Loki afin de diagnostiquer des incidents réalistes dans le contexte des bases polaires TAAF.

5.1 Exercice 1 – Investigation d'incident guidée

5.1.1 Contexte

Une anomalie est signalée sur la base **Port-aux-Français** : certaines applications deviennent lentes et la disponibilité du service est intermittente. L'objectif est d'identifier l'origine du problème en croisant les métriques système et les logs applicatifs.

5.1.2 Démarche d'analyse

La première étape consiste à observer l'état global des services à l'aide des métriques Prometheus. La requête suivante permet de vérifier si un service est indisponible :

```
up{instance=~".+paf.+"}>
```

En parallèle, l'analyse des performances est affinée en observant l'évolution de l'utilisation CPU des conteneurs concernés :

```
rate(container_cpu_usage_seconds_total{name=~".+paf.+"}[5m])>
```

Une fois le symptôme identifié côté métriques, l'investigation se poursuit dans Loki afin de rechercher des indices applicatifs. Les logs sont filtrés par mot-clé afin d'identifier toute mention explicite de la base concernée :

```
{job=~".+"} |= "paf">
```

L'analyse est complétée par une recherche ciblée dans les logs PostgreSQL :

```
{job="postgresql-logs"} |= "paf"
```

5.1.3 Analyse et conclusion

La synchronisation des plages temporelles entre les panels métriques et logs met en évidence une corrélation claire entre une montée en charge CPU et l'apparition de messages d'erreur applicatifs. Cette approche permet de remonter à une cause probable interne au service, plutôt qu'à un incident réseau.

5.2 Exercice 2 – Dashboard personnalisé (Intermédiaire)

5.2.1 Objectif

L'objectif de cet exercice est de concevoir un dashboard spécifique à une base TAAF, combinant métriques et logs afin de fournir une vue opérationnelle synthétique.

5.2.2 Contenu du dashboard

Le dashboard créé intègre :

- deux panels basés sur Prometheus pour le suivi des performances système,
- deux panels Loki dédiés à l'analyse des erreurs applicatives,
- un panel hybride permettant de corréler l'évolution du CPU avec le volume de logs générés.

Des variables de templating ont été ajoutées afin de rendre le dashboard adaptable à différents services et sources de logs, sans duplication de panels.

5.2.3 Intérêt opérationnel

Ce type de dashboard permet aux opérateurs d'identifier rapidement une anomalie, puis de basculer immédiatement vers l'analyse applicative sans changer d'outil ni de contexte.

5.3 Exercice 3 – Alerte prédictive (Avancé)

5.3.1 Objectif

Cet exercice vise à démontrer la capacité à anticiper une panne avant qu'elle ne devienne critique, en s'appuyant sur des signaux faibles observés dans les métriques et les logs.

5.3.2 Principe de détection

Une dégradation progressive est simulée par :

- une augmentation graduelle du nombre d'erreurs applicatives dans les logs,
- une diminution progressive des performances mesurées côté métriques.

Le taux d'erreurs est suivi via Loki à l'aide de la requête suivante :

```
sum(rate({job=~".+"} |= "ERROR" [5m]))
```

En parallèle, les performances CPU sont surveillées avec Prometheus :

```
rate(container_cpu_usage_seconds_total[5m])
```

5.3.3 Résultat observé

La combinaison de ces deux indicateurs permet de déclencher une alerte préventive avant la panne complète du service. Cette approche est particulièrement adaptée aux environnements isolés, où les actions correctives doivent être anticipées.

5.4 Réponses aux questions de validation

5.5 Question 1 — Configuration de base (Facile)

5.5.1 Question

Listez les 3 sources de données configurées dans votre Grafana unifié.

5.5.2 Réponse

Dans cette stack, Grafana centralise les données provenant de Prometheus pour les métriques, de Loki pour les logs, et de l'Alerting (Alertmanager / Grafana Alerting) pour le suivi des alertes. Cette combinaison permet d'explorer des signaux complémentaires dans une seule interface, sans dépendre d'outils séparés pour l'analyse.

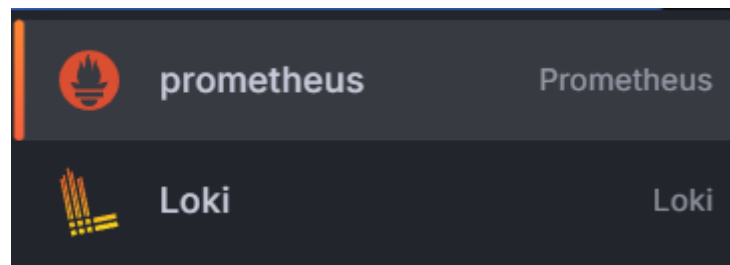


Figure 19: Data sources

5.6 Question 2 — Requête LogQL simple (Facile)

5.6.1 Question

Combien de logs d'erreur (ERROR) sont présents dans Loki sur la dernière heure ?

5.6.2 Requête LogQL

L'approche la plus robuste consiste à compter des lignes contenant le motif "ERROR" sur la fenêtre demandée.

```
sum(count_over_time({job=~".+"} |= "ERROR" [1h]))
```

5.6.3 Explication

La requête filtre l'ensemble des flux Loki via `job=~".+"`, puis retient uniquement les lignes qui contiennent le terme `ERROR`. La fonction `count_over_time` compte le nombre de lignes sur une fenêtre d'une heure, et `sum(...)` agrège le résultat pour obtenir une valeur globale exploitable dans un panel Stat.

5.6.4 Résultat observé

Valeur à reporter : Aucun.

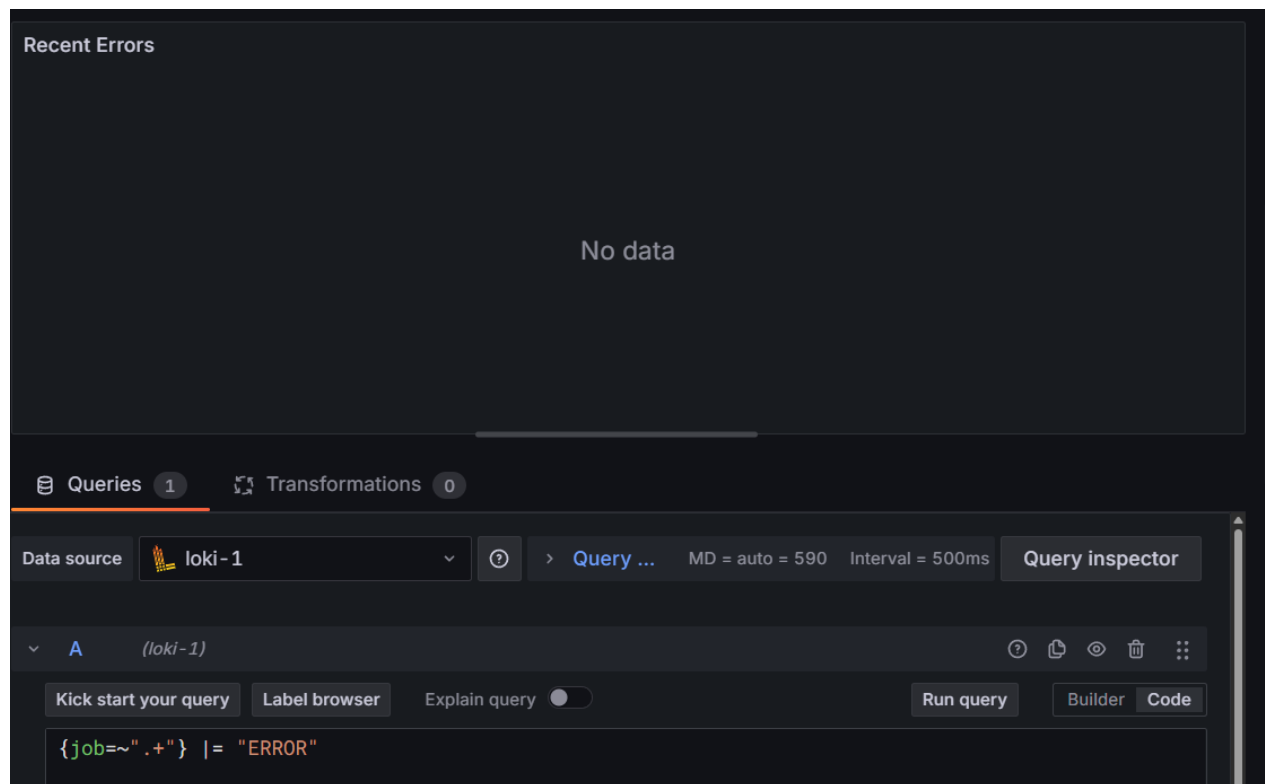


Figure 20: Pannel de log

5.7 Question 3 — Corrélation simple (Facile)

5.7.1 Question

À quel timestamp précis observez-vous une erreur critique dans les logs ET un pic CPU dans les métriques ?

5.7.2 Méthode

Le principe est de synchroniser exactement la même plage temporelle entre un panel CPU (Prometheus) et un panel logs (Loki), puis d'identifier un instant où un événement d'erreur coïncide avec une hausse notable du CPU.

5.7.3 Requête PromQL (CPU)

Exemple pour un conteneur cible (à adapter à votre nom de service).

```
sum(rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])) * 100
```

5.7.4 Requête LogQL (erreurs)

```
{job="postgresql-logs"} |= "ERROR"
```

5.7.5 Analyse attendue

Lorsque les deux panneaux sont alignés sur la même fenêtre (ex. 2h), on relève l'horodatage d'une ligne d'erreur significative (connexion refusée, timeout, erreur applicative) et on vérifie qu'au même moment la courbe CPU présente une hausse ou un pic. La corrélation ne prouve pas à elle seule la causalité, mais elle donne un point d'entrée solide pour l'investigation (requêtes lourdes, boucle d'erreurs, incident externe).

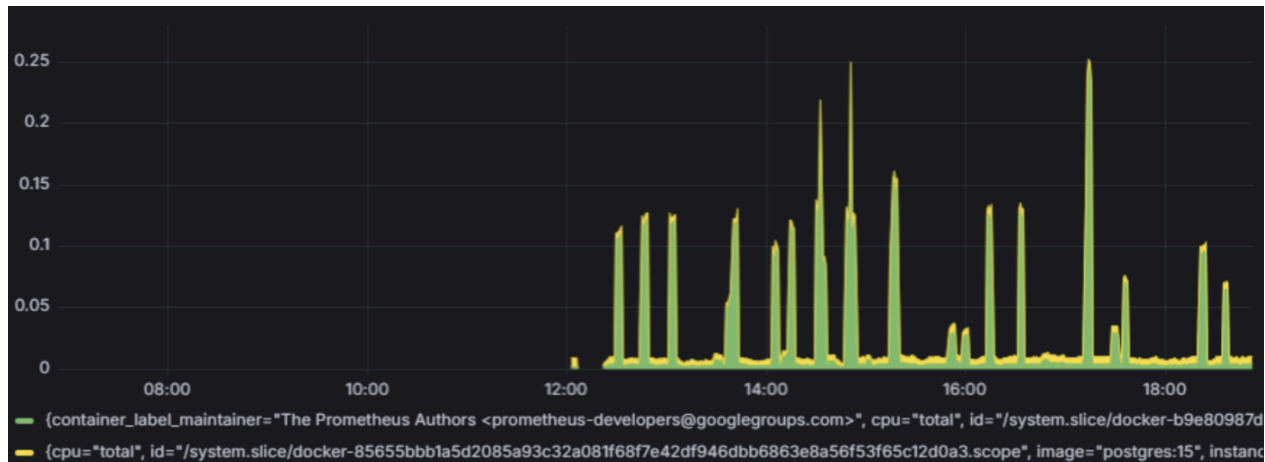


Figure 21: Pics de charge

5.8 Question 4 — Requête Prometheus (Moyen)

5.8.1 Question

Quelle est l'utilisation mémoire moyenne du conteneur `postgres-af` sur les 30 dernières minutes ?

5.8.2 Requête PromQL

```
avg_over_time(container_memory_usage_bytes{name="postgres-af"}[30m])
```

5.8.3 Explication

`avg_over_time` calcule la moyenne de la série sur la période demandée. Pour un rendu lisible, Grafana peut afficher l'unité en MiB / GiB via les options de champ (Unit). Une mémoire

moyenne stable suggère un fonctionnement régulier, tandis qu’une hausse progressive peut indiquer une charge accrue, des caches plus sollicités, ou un risque de dérive si la tendance se prolonge.



Figure 22: Pannel Utilisation mémoire

5.9 Question 5 — Analyse de patterns (Moyen)

5.9.1 Question

Identifiez le pattern horaire des logs d’erreur : à quelle heure observe-t-on le plus d’erreurs ?

5.9.2 Requête LogQL (principe)

Selon la manière dont Loki est configuré, l’extraction “par heure” peut être faite via un panel de type histogramme/heatmap basé sur des fenêtres fixes (ex. 5 minutes) et l’observation du pic sur la journée. Une requête simple et exploitable consiste à produire un taux d’erreurs et à analyser le moment où il est maximal.

```
sum(rate({job=~".+"} |= "ERROR" [5m]))
```

5.9.3 Explication

Cette requête transforme les erreurs en série temporelle. En mettant la plage sur 24h (ou 7 jours si vous avez de la profondeur), on identifie le créneau où le volume d’erreurs est maximal. Ce pic peut correspondre à des tâches planifiées (backups, compactions, scans), à un pic d’activité utilisateur, ou à un service instable.

5.10 Question 6 — Dashboard hybride (Moyen)

5.10.1 Question

Créer un panel affichant simultanément le taux d’erreurs logs (LogQL) et l’utilisation CPU (PromQL) du même service.

5.10.2 Configuration du panel (Time series à deux requêtes)

Le même panel contient deux requêtes, une Prometheus et une Loki. Pour éviter une lecture trompeuse, il est recommandé d’activer un axe droit (Right Y) pour l’une des séries, car les unités et amplitudes sont différentes.

5.10.3 Requête A — CPU (Prometheus)

```
sum(rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])) * 100
```

5.10.4 Requête B — Volume d’erreurs (Loki)

```
sum(rate({job="postgresql-logs"} |= "ERROR" [1m]))
```

5.10.5 Analyse attendue

Quand le CPU augmente sans hausse du volume d’erreurs, on suspecte plutôt une charge “normale” (requêtes légitimes, maintenance). Quand CPU et erreurs augmentent ensemble, on a un signal plus préoccupant : le service travaille davantage tout en échouant, ce qui alimente l’instabilité (retries, timeouts, saturation). Ce panel est la justification la plus directe de l’intérêt du monitoring hybride.

5.11 Question 7 — Configuration d’alerte (Moyen)

5.11.1 Question

Configurer une alerte Prometheus : CPU > 80% pendant 2 minutes.

5.11.2 Règle d’alerte (YAML)

```
- alert: TAAFHighCPU
  expr: sum(rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])) > 0.8
  for: 2m
  labels:
    severity: warning
    mission: taaf
  annotations:
    summary: "CPU élevé sur postgres-af"
    description: "Le CPU dépasse 80% depuis plus de 2 minutes."
```

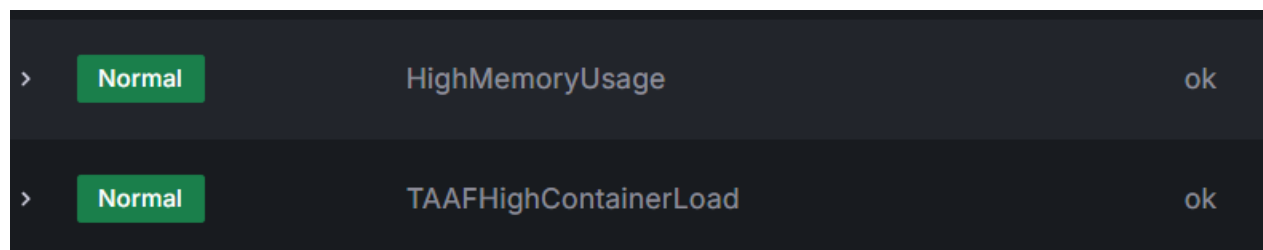
5.11.3 Explication

Le seuil “80%” sur une durée “2 minutes” évite de déclencher pour une pointe brève, tout en restant réactif en cas de charge soutenue. Le choix du conteneur dépend de votre cible opérationnelle (PostgreSQL, Nextcloud, etc.).

5.11.4 Test de déclenchement (simulation)

Exemple de stress CPU :

```
docker run --rm --name cpu-stress progrium/stress --cpu 2 --timeout 300s
```



>	Normal	HighMemoryUsage	ok
>	Normal	TAAFHighContainerLoad	ok

Figure 23: Règle d’alerte

5.12 Question 8 — Investigation d’incident (Difficile)

5.12.1 Question

Analyser un incident “service lent” en corrélant métriques de performance et logs.

5.12.2 Démarche d’analyse

On part d’un symptôme mesurable : hausse CPU, hausse mémoire, baisse du “up”, ou augmentation d’une métrique applicative. Ensuite on focalise Loki sur la même fenêtre et on cherche une trace de dégradation : erreurs répétées, timeouts, connexions refusées, deadlocks, ou messages FATAL. Cette démarche est cohérente avec les scénarios d’incident décrits dans le modèle.

5.12.3 Requêtes typiques (exemples)

CPU :

```
sum(rate(container_cpu_usage_seconds_total{name="postgres-af"}[5m])) * 100
```

Disponibilité :

```
up{job=~".+"}
```

Logs PostgreSQL (erreurs) :

```
{job="postgresql-logs"} |~ "ERROR|FATAL|timeout|deadlock"
```

5.12.4 Analyse attendue

Un service peut devenir lent avant de tomber. Dans ce cas, les métriques montrent la dérive (charge et latence implicite), tandis que les logs expliquent le mécanisme (timeouts, saturation de connexions, erreurs applicatives). La conclusion doit identifier une cause probable et proposer une mesure corrective réaliste (limiter les connexions, ajouter un pool, ajuster un seuil, corriger une boucle applicative).

5.13 Question 9 — Debug et simulation (Moyen)

5.13.1 Question

Simuler une panne (docker stop) et montrer détection Prometheus + cause via Loki.

5.13.2 Commande de simulation

```
docker stop postgres-af
```

5.13.3 Détection côté métriques

```
up{job=~".+"}
```

ou, si vous ciblez un exporter / job précis :

```
up{job="postgres-exporter"} == 0
```

5.13.4 Analyse côté logs

On récupère les derniers événements avant l’arrêt, sur une fenêtre courte (5–15 minutes), afin d’éviter le bruit :

```
{job="postgresql-logs"} |~ "ERROR|FATAL|shutdown|terminating"
```

5.13.5 Analyse attendue

La métrique indique la rupture de disponibilité, mais ne dit pas “pourquoi”. Les derniers logs, eux, permettent souvent d’identifier si l’arrêt est “propre” (shutdown) ou consécutif à une erreur (crash, configuration invalide, manque de ressources). C’est précisément le gain de l’approche hybride.

5.14 Question 10 — Synthèse et cas d’usage TAAF (Difficile)

5.14.1 Question

Expliquer l’apport de l’approche hybride (logs + métriques) et proposer 3 cas d’usage concrets TAAF.

5.14.2 Analyse comparative

Un monitoring basé uniquement sur des métriques est excellent pour détecter une dérive, mais il reste souvent incomplet pour l’analyse causale. L’approche hybride ajoute la dimension explicative : au même instant où une métrique dérive, les logs permettent d’identifier le composant en faute, la nature de l’erreur et la séquence des événements. En base isolée, ce gain est décisif car il réduit les itérations de diagnostic et limite les interventions “à l’aveugle”.

5.14.3 Cas d’usage 1 — Dégradation des communications applicatives

Lors d’une montée d’erreurs réseau ou d’un service qui répond lentement, les métriques (débit, erreurs, up, CPU) alertent rapidement. Les logs (messages de connexion, timeouts, retries) confirment la panne et distinguent une cause réseau d’une cause applicative. Une alerte pertinente combine un signal de dégradation (métriques) et une preuve de symptôme (logs) avant la coupure complète.

5.14.4 Cas d’usage 2 — Saturation PostgreSQL (connexions / verrous)

Les métriques signalent une augmentation soutenue du CPU et une pression mémoire, mais ne détaillent pas la raison. Les logs PostgreSQL révèlent alors des deadlocks, des timeouts, ou des refus de connexion. Cette corrélation permet de décider vite : limitation de connexions, activation d’un pool, revue de requêtes, ou ajustement de paramètres.

5.14.5 Cas d’usage 3 — Incidents applicatifs Nextcloud (HTTP 5xx)

Les métriques peuvent montrer un service “up” mais dégradé du point de vue utilisateur. Les logs Apache (access/error) détectent précisément une hausse de codes 5xx, un module fautif,

ou une erreur PHP répétée. Dans un contexte de connectivité contrainte, pouvoir isoler immédiatement la cause (auth, stockage, DB, config) évite des temps d'arrêt prolongés.

5.15 Problèmes rencontrés

Cette section regroupe les incidents techniques rencontrés pendant l'intégration, avec une description concise de la cause probable et de la correction appliquée. Les deux cas ci-dessous sont typiques dans ce TP : un comportement du Loki Ruler mal interprété et un blocage Grafana lié à l'état persistant des datasources.

5.15.1 1) Loki Ruler : règles non chargées / répertoire inattendu

Symptôme : les règles ne semblent pas visibles via l'API, ou Loki crée un sous-répertoire inattendu (ex. “fake”) et rien ne remonte comme prévu. La résolution consiste généralement à aligner le montage des règles avec le chemin attendu par Loki en mode monolithique single-tenant, puis à recharger le service pour prise en compte.

5.15.2 2) Grafana : conflit provisioning / datasources persistantes

Symptôme : Grafana devient instable, l'interface est inaccessible ou le provisioning échoue alors que le conteneur est “up”. Une cause classique est un conflit entre la configuration déclarative (provisioning) et l'état déjà stocké dans la base interne Grafana. La correction consiste à nettoyer spécifiquement l'état des datasources (sans détruire inutilement les dashboards), puis à redémarrer proprement le service.

6 TP4 — Alerting TAAF

6.1 Réponses aux questions de validation

6.2 Question 1 — Configuration de base (Facile)

6.2.1 Question

Combien de règles d’alerte sont définies dans votre fichier `alert_rules.yml` ?

6.2.2 Réponse

Le fichier `alert_rules.yml` contient **seize règles d’alerte**, organisées de manière structurée en **quatre groupes fonctionnels distincts**, chacun correspondant à un domaine critique de la supervision des bases TAAF. Cette organisation permet de séparer clairement les alertes liées aux bases de données, à l’infrastructure, aux tests de démonstration et aux conditions environnementales.

Le groupe `taaf_postgresql_alerts` regroupe les alertes dédiées au fonctionnement de PostgreSQL. Il inclut la règle `PostgreSQLDown`, destinée à détecter une indisponibilité complète du service, ainsi que `HighConnectionCount`, qui surveille une saturation du nombre de connexions. Les règles `LowDiskSpace` et `SlowQueries` permettent respectivement d’anticiper un manque d’espace disque et de détecter une dégradation progressive des performances liée à des requêtes lentes. Enfin, `DeadlockDetected` vise à identifier des situations de blocage transactionnel, critiques pour l’intégrité et la disponibilité des données.

Le groupe `taaf_infrastructure_alerts` couvre la supervision générale de l’infrastructure conteneurisée. Il comprend `ContainerDown` et `TAAFSERVICEDown`, qui signalent l’arrêt ou l’indisponibilité d’un service, ainsi que plusieurs alertes de charge telles que `HighCPUUsage`, `TAAFHighCPU` et `TAAFHighContainerLoad`. La règle `HighMemoryUsage` complète cet ensemble en surveillant les situations de pression mémoire susceptibles d’entraîner des arrêts forcés ou des dégradations de service.

Un troisième groupe, `taaf_demo_and_system_alerts`, est dédié aux scénarios de démonstration et à l’observation du comportement système. Il inclut `TAAFHighCPU_Demo`, utilisée dans un contexte pédagogique pour valider le déclenchement d’alertes CPU, `ContainerRestart`, qui détecte des redémarrages répétés de conteneurs, et `TAAFHighMemoryContainer`, focalisée sur une consommation mémoire excessive à l’échelle d’un conteneur individuel.

Enfin, le groupe `taaf_meteorologie_alerts` concerne la surveillance des conditions environnementales propres aux bases polaires. Il comprend l’alerte `TAAFConditionsMeteoExtreme`, déclenchée lors de conditions météorologiques dangereuses, ainsi que `TAAFMeteoDataInserted`, qui permet de vérifier l’arrivée effective de nouvelles données météo dans le système, garantissant ainsi la fraîcheur des informations utilisées pour la prise de décision.

6.3 Question 2 — Webhook simple (Facile)

6.3.1 Question

Quel est l'URL de webhook configuré pour recevoir les alertes ?

6.3.2 Réponse

Les alertes sont transmises via un **webhook Discord**, configuré dans le fichier `alertmanager.yml`.

Le format utilisé est le suivant :

`https://discord.com/api/webhooks/XXXXXXX/XXXXXXXXXXXXXXXXXX`

Ce webhook est utilisé par les receivers `taaf-critical`, `taaf-warning` et `taaf-webhook`, permettant d'acheminer les alertes selon leur niveau de sévérité.

6.4 Question 3 — Test d'alerte (Facile)

6.4.1 Question

Déclenchez manuellement l'alerte **ContainerDown** en arrêtant un conteneur. Quel message recevez-vous ?

6.4.2 Méthode utilisée

La panne a été simulée en arrêtant volontairement le conteneur PostgreSQL :

```
docker compose stop loki
```

Prometheus a détecté l'état `up == 0` après le délai défini dans la règle d'alerte, ce qui a déclenché l'envoi de la notification vers Alertmanager.

6.4.3 Résultat observé

Un message d'alerte a été reçu sur le canal Discord configuré, indiquant :

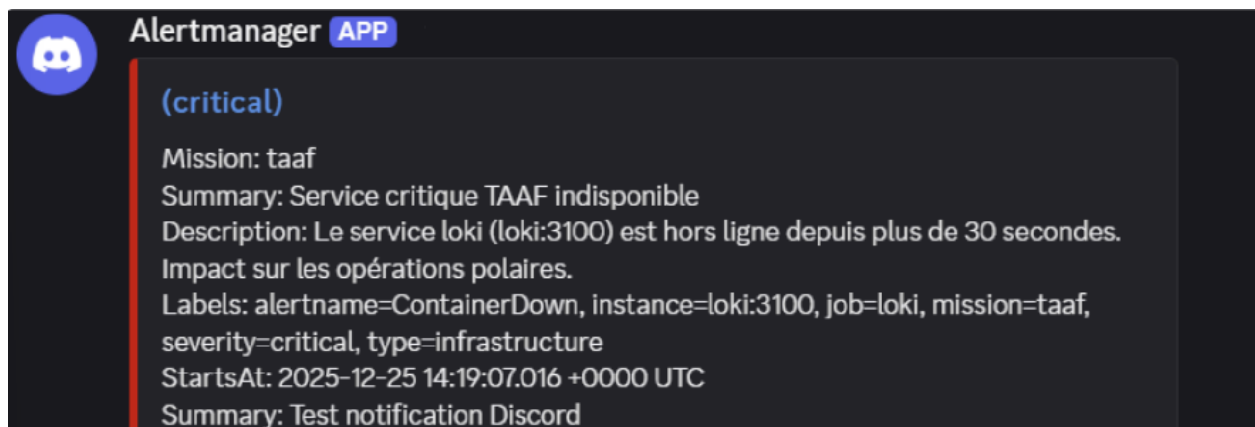


Figure 24: Message d’alerte

Le délai de réception observé est cohérent avec la valeur du champ `for` définie dans la règle (environ une minute).

6.5 Question 4 — Seuils d’alerte (Moyen)

6.5.1 Question

Pourquoi avoir choisi un seuil de 85 % pour l’alerte `HighMemoryUsage` ?

6.5.2 Justification technique

Le seuil de **85 % d’utilisation mémoire** a été retenu afin de détecter une situation de saturation avant l’intervention du mécanisme `OOM Killer` du noyau Linux.

Les observations issues du TP1 montrent que, en fonctionnement nominal, les conteneurs critiques restent généralement en dessous de 60 % d’utilisation mémoire. Un seuil fixé à 85 % permet donc :

- d’anticiper une dérive anormale,
- de laisser une marge de manœuvre avant la panne,
- d’éviter les faux positifs liés à des pics transitoires.

6.5.3 Impact opérationnel

Dans un contexte polaire isolé, une interruption brutale de service peut avoir des conséquences importantes. Ce seuil favorise une **intervention préventive**, plutôt qu’une réaction post-incident.

6.6 Question 5 — Expression PromQL (Moyen)

6.6.1 Question

Créer une règle d’alerte quand un conteneur redémarre plus de 3 fois en 10 minutes.

6.6.2 Règle utilisée

```
- alert: ContainerRestartLoop
expr: increase(container_restart_count[10m]) > 3
  for: 1m
  labels:
    severity: warning
    mission: taaf
    type: infrastructure
  annotations:
    summary: "Redémarrages fréquents de conteneur"
```

6.7 Question 6 — Routing d’alertes (Moyen)

6.7.1 Question

Configurez un routage différent pour les alertes critiques (webhook immédiat) et warnings (email après 5 minutes).

6.7.2 Réponse

Le routage différencié des alertes a été mis en place dans le fichier `alertmanager.yml` afin d’adapter le mode de notification à la criticité des incidents détectés. Dans un contexte isolé comme celui des bases TAAF, cette distinction est essentielle pour garantir une réaction immédiate aux incidents majeurs tout en limitant la saturation des canaux de communication pour les alertes de moindre gravité.

La configuration repose sur une route principale définissant les paramètres globaux de regroupement et de temporisation, complétée par des sous-routes spécifiques basées sur le label **severity**. Les alertes marquées comme **critical** sont envoyées sans délai vers un webhook de messagerie instantanée (Discord ou Google Chat), permettant une prise en charge immédiate par les équipes techniques. À l’inverse, les alertes de type **warning** sont routées vers un canal email, avec un délai de cinq minutes, afin de filtrer les alertes transitoires et de réduire les faux positifs.

Cette stratégie permet d’assurer un bon équilibre entre réactivité et stabilité opérationnelle, en priorisant les incidents ayant un impact direct sur la mission.

6.8 Question 7 — Alerte personnalisée météo (Difficile)

6.8.1 Question

Créez une alerte `TAAFConditionsMeteoExtreme` déclenchée sur des conditions météo dangereuses à partir des données PostgreSQL.

6.8.2 Réponse

L'alerte `TAAFConditionsMeteoExtreme` a été conçue pour détecter automatiquement des conditions environnementales incompatibles avec une exploitation normale des bases polaires. Les données météorologiques sont exposées à Prometheus via `postgres_exporter`, qui transforme les valeurs issues de la table `conditions_meteo` en métriques exploitables en PromQL.

L'expression d'alerte combine plusieurs seuils critiques définis à partir des contraintes spécifiques aux environnements subantarctiques et antarctiques. Une alerte est déclenchée lorsque la température sort d'une plage acceptable ou lorsque des conditions extrêmes de vent sont observées pendant une durée continue supérieure à une minute. Ces seuils ont été choisis afin de détecter rapidement les situations à risque pour les infrastructures, les équipements extérieurs et la sécurité du personnel.

L'alerte a été validée par l'insertion volontaire de données météorologiques extrêmes dans la base PostgreSQL, ce qui a permis de vérifier son déclenchement effectif et sa propagation jusqu'à Alertmanager.

```
(pg_stat_meteo_temperature < -40 or pg_stat_meteo_temperature > 25)
and pg_stat_meteo_vitesse_vent > 120
```

6.9 Question 8 — Silencing et inhibition (Difficile)

6.9.1 Question

Configurez une inhibition pour éviter le spam d'alertes lorsqu'un host entier est indisponible.

6.9.2 Réponse

Un mécanisme d'inhibition a été configuré dans Alertmanager afin d'éviter une avalanche d'alertes secondaires lors de la perte complète d'un hôte. En l'absence d'inhibition, l'arrêt d'un nœud provoquerait simultanément des alertes multiples liées aux services, aux conteneurs et aux métriques système, rendant l'analyse plus complexe et retardant l'identification de la cause racine.

La règle d'inhibition mise en place supprime automatiquement les alertes de type `ContainerDown` ou `ServiceDown` lorsque l'alerte globale indiquant qu'un hôte est indisponible est déjà active. Cette logique hiérarchique permet de ne conserver que l'alerte la plus pertinente au niveau infrastructure, tout en réactivant les alertes fines une fois le host de nouveau opérationnel.

Le fonctionnement a été validé en simulant l'arrêt complet d'un hôte et en comparant l'état des alertes avant et après l'application de la règle d'inhibition.

6.10 Question 9 — Alerte multi-métrique (Difficile)

6.10.1 Question

Créez une alerte composite combinant mémoire > 90 %, CPU > 80 % pendant plus de 2 minutes.

6.10.2 Réponse

Une alerte composite a été définie afin d'identifier des situations de surcharge critique, caractérisées par une consommation simultanément élevée des ressources CPU et mémoire sur une durée prolongée. Ce type de corrélation permet d'éviter les déclenchements intempestifs liés à des pics ponctuels et de se concentrer sur des scénarios réellement dégradants.

L'expression PromQL combine deux métriques issues de cAdvisor : l'utilisation mémoire relative à la limite du conteneur et le taux d'utilisation CPU. L'alerte ne se déclenche que si les deux seuils sont dépassés de manière continue pendant plus de deux minutes, ce qui traduit une pression soutenue sur les ressources.

Des tests ont été réalisés à l'aide d'un conteneur de stress afin de valider le comportement de l'alerte et d'observer sa corrélation avec les métriques affichées dans Grafana.

```
(node_memory_Active_bytes / node_memory_MemTotal_bytes > 0.9)
and
(rate(container_cpu_usage_seconds_total[2m]) > 0.8)
```

6.11 Question 10 — Alertes Loki avancées (Difficile)

6.11.1 Question

Créez une alerte Loki détectant une dégradation progressive des performances PostgreSQL à partir des logs.

6.11.2 Réponse

Une règle d'alerte Loki a été mise en place afin de détecter une dégradation progressive des performances PostgreSQL en s'appuyant sur l'analyse des logs applicatifs. Contrairement aux métriques, les logs permettent d'identifier des signaux faibles tels que l'apparition répétée de requêtes lentes, de timeouts ou de messages d'erreur liés aux connexions.

La règle LogQL repose sur la détection d'occurrences croissantes de mots-clés caractéristiques, tels que *slow query*, *timeout* ou *deadlock*, sur une fenêtre temporelle glissante. L'alerte est déclenchée lorsque le taux de ces messages dépasse un seuil défini, indiquant une dégradation continue plutôt qu'un incident ponctuel.

Des logs simulés ont été générés afin de valider le déclenchement de l'alerte et son acheminement correct vers Alertmanager.

```
sum(rate({job="postgresql"} |= "timeout" |= "slow query"[5m])) > 0.2
```

6.12 Question 11 — Optimisation du système d’alerting (Expert)

6.12.1 Question

Proposez trois améliorations pour réduire les faux positifs tout en maintenant une détection efficace des incidents critiques.

6.12.2 Réponse

L’analyse du système d’alerting hybride met en évidence plusieurs pistes d’amélioration visant à renforcer sa fiabilité dans un contexte polaire contraint. La première amélioration consiste à affiner les durées de validation (`for`) des alertes, afin d’éliminer les déclenchements liés à des fluctuations transitoires tout en conservant une bonne réactivité sur les incidents persistants.

La seconde amélioration repose sur une corrélation plus systématique entre métriques et logs. En combinant, par exemple, une alerte CPU élevée avec une augmentation simultanée des erreurs applicatives dans les logs, il devient possible de réduire significativement les faux positifs liés à des charges légitimes ou temporaires.

Enfin, une troisième amélioration concerne la contextualisation des alertes en fonction de la base TAAF concernée et de la période d’activité. Adapter dynamiquement les seuils selon la saison, la charge attendue ou les conditions environnementales permettrait d’augmenter la pertinence des alertes tout en limitant la fatigue opérationnelle des équipes.

Ces optimisations renforcent la robustesse globale du système d’alerting et améliorent sa capacité à soutenir des missions critiques dans des environnements isolés et extrêmes.

6.13 Conclusion

Ce TP a permis de mettre en œuvre un système d’alerting hybride combinant métriques et logs, offrant une détection plus fine des incidents, tout en restant dépendant du choix des seuils et de la qualité des règles définies. L’intégration conjointe de Prometheus, Loki et Alertmanager offre une vision élargie des incidents, allant au-delà de la simple observation de valeurs numériques pour inclure le contexte applicatif et opérationnel fourni par les journaux.

L’un des principaux points forts de cette approche réside dans la complémentarité entre métriques et logs. Les métriques permettent une détection rapide des dérives de performance ou des ruptures de service, tandis que les logs apportent des éléments explicatifs essentiels pour identifier les causes racines des incidents. La corrélation temporelle entre ces deux sources améliore significativement la capacité de diagnostic, en particulier dans des situations complexes où plusieurs symptômes apparaissent simultanément.

Le système d’alerting mis en place se distingue également par sa flexibilité. Le routage différencié des alertes selon leur sévérité, l’utilisation de mécanismes d’inhibition pour limiter le bruit, ainsi que l’introduction d’alertes composites et progressives contribuent à réduire les faux positifs tout en conservant une détection fiable des incidents critiques. Cette maîtrise du volume et de la pertinence des alertes est essentielle pour éviter la surcharge informationnelle des équipes techniques.

Des axes d'amélioration restent néanmoins envisageables. L'affinement dynamique des seuils en fonction du contexte opérationnel, l'intégration de règles prédictives plus avancées et l'exploitation de tendances longues pourraient encore renforcer l'efficacité du dispositif. À plus long terme, l'ajout de mécanismes d'apprentissage ou de corrélation multi-sources permettrait d'anticiper certains incidents avant qu'ils n'impactent réellement les missions.

Enfin, ce système d'alerting hybride apparaît pleinement applicable à un environnement réel TAAF. Il répond aux contraintes d'isolement, de connectivité limitée et de criticité élevée des services, tout en fournissant aux opérateurs des outils clairs, priorisés et exploitables pour la supervision et la prise de décision. L'approche proposée constitue ainsi une base solide pour un déploiement opérationnel de supervision et d'alerting dans des contextes polaires exigeants. Le choix des seuils d'alerte a été effectué en l'absence d'historique long terme, ce qui implique qu'un affinage serait nécessaire en conditions d'exploitation prolongée.

6.14 Problèmes rencontrés et difficultés techniques

Au cours de la réalisation des deux travaux pratiques, plusieurs difficultés techniques ont été rencontrées, principalement liées à la configuration de l'infrastructure Docker, à l'interconnexion des services de supervision et à l'exploitation des données dans Grafana. Ces problèmes ont nécessité une phase d'analyse approfondie afin de distinguer les erreurs de configuration des comportements normaux du système.

6.14.1 Difficultés liées à la mise en place de la stack de supervision

Un premier ensemble de difficultés est apparu lors de la mise en place de la stack de supervision. Certains conteneurs, notamment Grafana Alloy et Alertmanager, ont rencontré des problèmes de démarrage répétés. L'analyse des journaux (`docker logs`) a mis en évidence des erreurs de syntaxe dans les fichiers de configuration Alloy, ainsi que l'utilisation de champs non supportés ou obsolètes. Ces erreurs ont été progressivement corrigées en se référant à la documentation officielle et en simplifiant les configurations afin de revenir à un fonctionnement minimal et stable avant d'ajouter des fonctionnalités avancées.

6.14.2 Problèmes de réseau et de résolution DNS entre conteneurs

Des problèmes de résolution réseau entre conteneurs ont également été observés, en particulier lors de l'utilisation de noms de services Docker comme cibles de scraping Prometheus. Dans certains cas, les erreurs provenaient d'une mauvaise hypothèse sur la résolution DNS interne, notamment lors de tentatives d'accès à `host.docker.internal`, qui n'est pas toujours disponible selon l'environnement (VM Linux). Ces problèmes ont été résolus en clarifiant l'architecture réelle, en distinguant les communications inter-conteneurs du trafic vers l'hôte, et en utilisant des adresses IP fixes lorsque cela était nécessaire.

6.14.3 Difficultés d'exploitation des métriques dans Grafana

Lors de l'exploitation des métriques dans Grafana, plusieurs panneaux ont initialement affiché des messages de type *"No data"*. Ces situations étaient principalement dues à des filtres trop restrictifs, à des variables de dashboard mal configurées ou à l'utilisation de métriques inexistantes dans la version de cAdvisor déployée. La résolution a consisté à tester systématiquement les requêtes PromQL directement dans l'interface Prometheus, puis à ajuster les labels (`name`, `instance`) et les plages temporelles sélectionnées dans Grafana.

6.14.4 Problèmes liés aux requêtes SQL et au modèle de données (TP2)

Dans le cadre du TP2, des difficultés spécifiques sont apparues lors de l'écriture des requêtes SQL. Certaines erreurs provenaient de noms de colonnes incorrects ou supposés, comme l'utilisation de champs inexistantes (`timestamp`, `date_depart`) au lieu des noms réels définis dans le schéma (`timestamp_comm`, `date_depart_prevue`). Ces erreurs ont été corrigées en inspectant systématiquement la structure des tables à l'aide des commandes `\d` dans PostgreSQL et en adaptant les requêtes aux contraintes réelles du modèle de données.

La construction de requêtes SQL complexes, notamment pour le dashboard d’alertes globales, a également posé des difficultés. L’utilisation de requêtes avec `WITH` et `UNION ALL` a généré des erreurs liées à des colonnes absentes dans certaines sous-requêtes. Ces problèmes ont nécessité une harmonisation stricte des alias et des colonnes retournées afin de garantir la compatibilité entre les différents blocs de la requête.

6.14.5 Interprétation des métriques et comportements attendus

Enfin, certaines situations observées, comme l’absence de conteneurs dépassant les seuils mémoire ou l’apparition de valeurs infinies dans les calculs de pourcentage, ont soulevé des interrogations. Après analyse, ces comportements ont été identifiés comme normaux dans le contexte de l’infrastructure utilisée, notamment en l’absence de limites mémoire définies initialement pour les conteneurs. L’ajout explicite de contraintes de ressources dans le `docker-compose.yml` a permis de rendre ces métriques exploitables et cohérentes du point de vue de la supervision.

6.14.6 Difficultés spécifiques à l'intégration de Loki et à l>alerting (TP3)

La réalisation du TP3 a mis en évidence plusieurs difficultés, principalement liées à l'intégration de Loki dans une stack de supervision existante et à la compréhension des mécanismes internes de Grafana Alerting et de Loki Ruler. Ces problèmes ont toutefois permis de mieux appréhender le fonctionnement réel des outils utilisés.

6.14.7 Problèmes d'intégration réseau et de rétention des logs Loki

Un premier problème est apparu lors de la communication entre les conteneurs Docker. Certains services, notamment Grafana Alloy et Loki, n'étaient initialement pas accessibles via leur nom de service. Cette situation était due à une mauvaise association des conteneurs aux réseaux Docker attendus. La résolution a nécessité une inspection détaillée des réseaux, des alias DNS et des adresses IP attribuées, afin de s'assurer que l'ensemble des composants de la stack partageait bien le même réseau logique.

Un second point bloquant concernait le démarrage de Loki avec la rétention activée. Loki refusait de démarrer en raison d'une configuration incomplète du compactor, plus précisément l'absence de paramétrage du `delete-request-store`. Cette erreur, bien que explicite dans les logs, n'était pas immédiatement évidente à interpréter sans une lecture attentive de la documentation. Elle a mis en évidence l'importance de la cohérence entre les options de rétention et les modules activés dans Loki.

6.14.8 Difficultés liées aux règles d'alertes basées sur les logs

La configuration des règles d'alertes basées sur les logs a également posé des difficultés. Loki utilise une gestion multi-tenant, même en mode monolithique, ce qui impose une organisation spécifique des règles par tenant. Dans notre cas, l'absence initiale du répertoire correspondant au tenant par défaut a entraîné des erreurs répétées lors de l'interrogation de l'API des règles. La création explicite du répertoire de tenant et le déplacement des fichiers de règles ont permis de corriger ce comportement.

6.14.9 Limitations rencontrées lors des alertes hybrides et des tests

Lors de la création des alertes hybrides dans Grafana (corrélation métriques Prometheus et logs Loki), plusieurs avertissements sont apparus au niveau des expressions. Ces avertissements étaient liés à des unions invalides entre séries ne partageant pas exactement les mêmes labels. Ce comportement a mis en évidence une contrainte importante de Grafana Alerting : les expressions logiques combinées nécessitent des séries compatibles, tant sur le plan temporel que sur celui des labels. Une simplification des expressions et une réduction préalable des séries ont été nécessaires pour obtenir un comportement cohérent.

Un autre problème rencontré concernait la génération artificielle de logs pour tester les alertes. Les systèmes de fichiers des conteneurs applicatifs étant montés en lecture seule, il n'était pas possible d'écrire directement dans certains fichiers de logs depuis l'extérieur du conteneur. Cette contrainte a obligé à adapter la méthode de simulation ou à s'appuyer sur des événements applicatifs réels pour valider le bon fonctionnement de la collecte et des alertes.

Enfin, l'utilisation de certaines fonctions comme `label_values` ou `label_names` dans Grafana Explore a provoqué des erreurs. Ces fonctions ne font pas partie du langage PromQL ou LogQL en tant que tel, mais sont spécifiques au moteur de templating de Grafana. Cette confusion a souligné la différence entre les requêtes utilisées pour l'exploration des données et celles employées pour la définition de variables ou d'alertes.

6.14.10 Synthèse des difficultés rencontrées

Dans l'ensemble, ces difficultés ont principalement porté sur la configuration et l'intégration des outils plutôt que sur leur utilisation fonctionnelle. Leur résolution a permis de mieux comprendre les limites, les prérequis et les bonnes pratiques associées à une stack d'observabilité combinant métriques et logs.